University of Strathclyde

CS411

Theory of Computation

2026, March 23rd

Rin

Contents

Table of Contents **i**

1 Sheet 1 **1**

2 Sheet 2 **7**

3 Sheet 3 **18**

4 Sheet 4 **27**

5 Sheet 5 **32**

6 Sheet 6 **38**

7 Sheet 7 **44**

1 Sheet 1

Q1

Let $L_1 = \{aa,bb,bbb\}$ and $L_2 = \{abba,aab,bb\}$ be languages over $\Sigma = \{a,b\}$. Compute the following languages:

(*i*) $L_1 \circ L_2$;

(*ii*) $L_1 \cup L_2$;

(*iii*) $L_1 \cap L_2$;

(*iv*) and provide a few examples of strings/words contained in L_2^*.
(Why am I not asking you to list all words?)

(*i*) The concatenation of L_1 and L_2 is the set $\{a \cdot b : a \in L_1, b \in L_2\}$. In this case:

$$L_1 \circ L_2 = \{aaabba,\ aaaab,\ aabb,\quad bbabba,\ bbaab,\ bbbb,\quad bbbabba,\ bbbaab,\ bbbbb\}$$

(*ii*)

$$L_1 \cup L_2 = \{aa,bb,bbb,abba,aab\}$$

(*iii*)

$$L_1 \cap L_2 = \{bb\}$$

(*iv*)

$$L_2^* = \{\varepsilon,abba,aab,bb,abbaabba,abbaaab,abbabb,aababba,aabaab,aabbb,\ldots\}$$

Recall that the Kleene star of a set X is defined as

$$\bigcup_{i \in \mathbb{N}} X^i$$

where the X^i are defined recursively by:

$$X^0 := \{\varepsilon\}$$
$$X^{i+1} := X^i \circ X$$

In other words:

$$X^* = \{\varepsilon\} \cup X \cup (X \circ X) \cup (X \circ X \circ X) \cup \cdots$$

So in general, for non-trivial countable X, X^* is (countably) infinite.

Q2

Give the state diagram for the DFAs accepting the following languages:

(*i*) $L_1 = \{w \in \{1,22\}^* \mid 11 \text{ is a suffix of } w\}$

(*ii*) $L_2 = \varnothing \subseteq \{0,1,2\}^*$

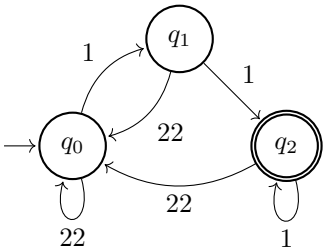
(*iii*) $L_3 = \{\varepsilon\} \subseteq \{0,1,2\}^*$

(*iv*) $L_4 = \{w \in \{\text{stop}, \text{go}\}^* \mid w = \varepsilon \text{ or } w \text{ ends with a stop}\}$

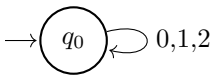
(*v*) $L_5 = \{w = \{0,1\}^* \mid w \text{ has 001 as a prefix or 11 as a suffix}\}$

Recall that a word x is a *prefix* (respectively, *suffix*) of y if there exists a word z such that $xz = y$ (resp. $zx = y$).

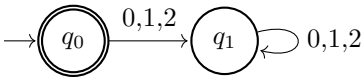
(*i*) We need three states to track the number of 1s that we have read so far, moving along each time we read a 1, and moving back to the first state whenever we read a 22:



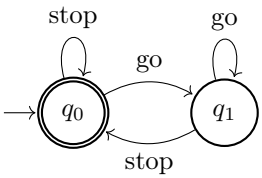
(*ii*) To recognise the empty language, we can just not include any accepting states in the DFA:



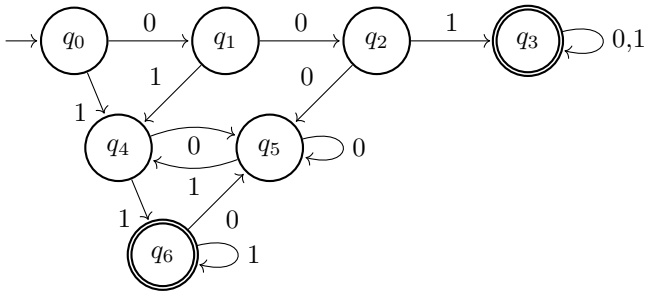
(*iii*) To accept ε, we make the initial state accepting, and to reject every other word, we immediately move to a dead state upon reading any other symbol:



(*iv*) We need two states to keep track of which of the two symbols was last read:



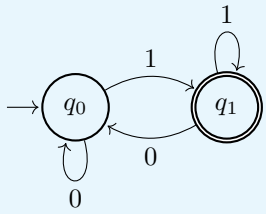
(*v*) We can immediately check for the prefix and go into an looping accepting state if it is matched, or otherwise match on the suffix at each exit point:



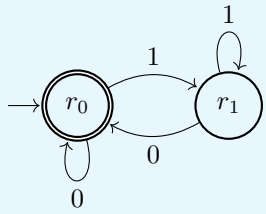
$Q3$

Consider the following automata:

M_1 :



M_2 :



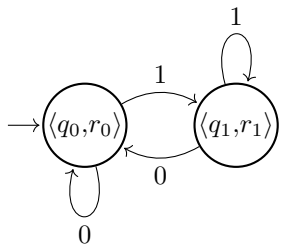
(i) Construct an automaton accepting $L(M_1) \cap L(M_2)$.

(ii) Prove that regular languages are closed under intersection.

(iii) Construct an automaton for the language $\{0,1\}^* \setminus \big(L(M_1) \cap L(M_2)\big)$.

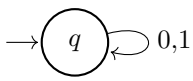
(iv) Prove that regular languages are closed under complementation.

(i) By pairing up the states, we have:



(the other state pairs are not reachable from $\langle q_0,r_0\rangle$.)

Alternatively, we could notice that the two languages are disjoint, and just give the DFA for the empty language:



(ii) The idea is to run both automata at the same time by operating on pairs of states, where a transition $(q,r) \xrightarrow{a} (q',r')$ exists if and only if both $q \xrightarrow{a} q'$ and $r \xrightarrow{a} r'$ exist in the two original automata.

Let L_1 and L_2 be regular, with $L_i = L(M_i)$ for DFA $M_i = (Q_i,\Sigma,\delta_i,q_0^i,F_i)$. Construct the automata $M = (Q,\Sigma,\delta,q_0,F)$ by:

$$
\begin{aligned}
Q &\coloneqq Q_1 \times Q_2 \\
\delta &\coloneqq \delta_1 \times \delta_2 \\
\delta\big(\langle q_1,q_2\rangle,a\big) &= \langle \delta_1(q_1,a),\delta_2(q_2,a)\rangle \\
q_0 &\coloneqq \langle q_0^1,q_0^2\rangle \\
F &\coloneqq F_1 \times F_2
\end{aligned}
$$

Then, $L(M) = L(M_1) \cap L(M_2)$. This is because, any accepting run of M on $w = a_0 \cdots a_n$ consists of accepting run of M_1 in the first component, and an accepting run of M_2 in the second:

$$M: \ \langle q_0^1,q_0^2\rangle \xrightarrow{a_0} \langle q_1^1,q_1^2\rangle \xrightarrow{a_1} \langle q_1^1,q_1^2\rangle \to \cdots \to \langle q_n^1,q_n^2\rangle \xrightarrow{a_n} \langle q_{n+1}^1,q_{n+1}^2\rangle$$

$$\updownarrow$$

$$M_1: \ q_0^1 \xrightarrow{a_0} q_1^1 \xrightarrow{a_1} q_1^1 \to \cdots \to q_n^1 \xrightarrow{a_n} q_{n+1}^1$$

and

$$M_2: \ q_0^2 \xrightarrow{a_0} q_1^2 \xrightarrow{a_1} q_1^2 \to \cdots \to q_n^2 \xrightarrow{a_n} q_{n+1}^2$$

Formally by induction:

Claim. *For all words $s \in \Sigma^*$,*

$$\hat{\delta}\big(q_0,s\big) = \big\langle \hat{\delta}_1(q_0^1,s),\hat{\delta}_2(q_0^2,s)\big\rangle$$

Proof. By induction on $|s|$. If $|s| = 0$, then $s = \varepsilon$, so:

$$\hat{\delta}(q,\varepsilon) = q = \langle q_0^1,q_0^2\rangle = \big\langle \hat{\delta}_1(q_0^1,\varepsilon),\hat{\delta}_2(q_0^2,\varepsilon)\big\rangle$$

Otherwise, suppose $s = wa$ with $w \in \Sigma^*$ and $a \in \Sigma$:

$$\hat{\delta}\big(\langle q_0^1,q_0^2\rangle,wa\big) = \delta\big(\hat{\delta}(\langle q_0^1,q_0^2\rangle,w), a\big)$$

By the induction hypothesis, the run of M on words shorter than wa agrees with the runs of M_1 and M_2. In particular, w is shorter than wa, so $\hat{\delta}(\langle q_0^1,q_0^2\rangle,w) = \big\langle \hat{\delta}_1(q_0^1,w),\hat{\delta}_2(q_0^2,w)\big\rangle$, which gives:

$$\begin{aligned} &= \delta\Big(\big\langle \hat{\delta}_1(q_0^1,w),\hat{\delta}_2(q_0^2,w)\big\rangle, a\Big) \\ &= \big\langle \hat{\delta}_1(q_0^1,wa),\hat{\delta}_1(q_0^2,wa)\big\rangle \end{aligned}$$

■

Now, M accepts a string s if and only if $\hat{\delta}\big(\langle q_0^1,q_0^2\rangle,s\big) \in F$ (that is, its run on s is accepting). By the above, this is equivalent to $\big\langle \hat{\delta}_1(q_0^1,s),\hat{\delta}_1(q_0^2,s)\big\rangle \in F_1 \times F_2$, which holds if and only if both components hold. That is, $\hat{\delta}_1(q_0^1,s) \in F_1$ (M_1 accepts s) and $\hat{\delta}_2(q_0^2,s) \in F_2$ (M_2 accepts s). So M accepts s if and only if M_1 and M_2 accept s.

(*iii*) To take the complement of the language, we can invert the accepting and rejecting states of the previous automaton:

$$\overline{M} = (Q, \Sigma, \delta, q_0, Q \setminus F)$$

(*iv*) This trick works in general: if $M = (Q,\Sigma,\delta,q_0,Q)$ accepts L, then

$$\overline{M} = (Q,\Sigma,\delta,q_0,Q \setminus F)$$

accepts $\Sigma^* \setminus L$.

Q4

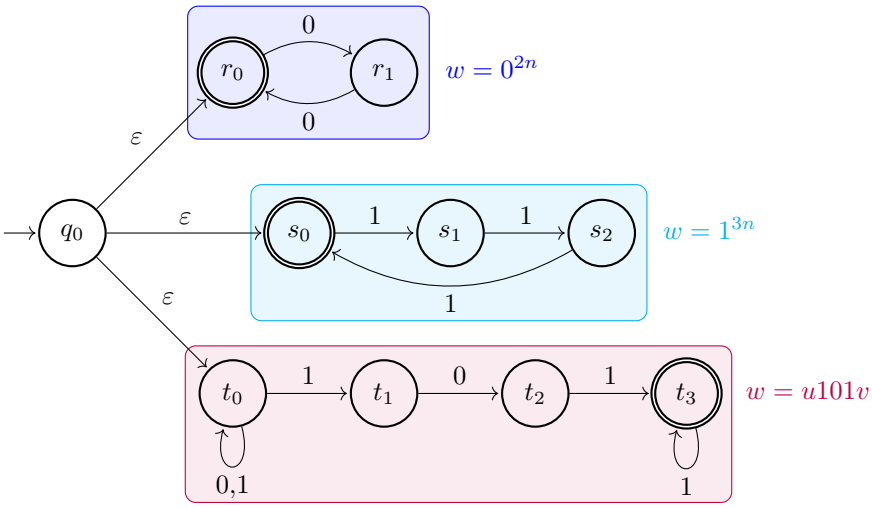
Construct finite automata that recognise the following languages over $\Sigma = \{0,1\}$:

(*i*) $L_1 = \{w \mid w = 0^{2n} \text{ or } w = 1^{3n} \text{ or } w = u101v \text{ for some } u,v \in \Sigma^*\}$

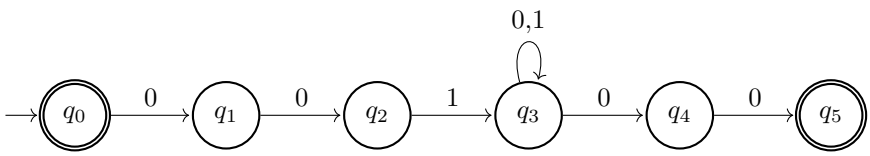
(*ii*) $L_2 = \{w \mid w \text{ has prefix 001 and suffix 00}\} \cup \{\varepsilon\}$

Hint: you may want to construct NFA rather than DFA.

(*i*) This is the union of three languages, so we can just construct the three automata separately, then connect to their starting states with ε-transitions:



(*ii*) We can immediately check for the prefix, have a looping state, then check for the suffix using non-determinism:



*Q*5

For a word $w \in \Sigma^*$, its *reverse* $(w)^r$ is defined recursively by:

$$(\varepsilon)^r \coloneqq \varepsilon$$
$$(wa)^r \coloneqq a(w)^r$$

(*i*) Let $\Sigma = \{a,b,c\}$. Prove that $(abc)^r = cba$ by unfolding the definition.

(*ii*) For a language L, we define the *reverse language* L^r by

$$L^r \coloneqq \left\{(w)^r \mid w \in L\right\}$$

Prove that L^r is regular if L is regular.

(*i*) $(abc)^r = c(ab)^r = cb(a)^r = cba(\varepsilon)^r = cba\varepsilon = cba$.

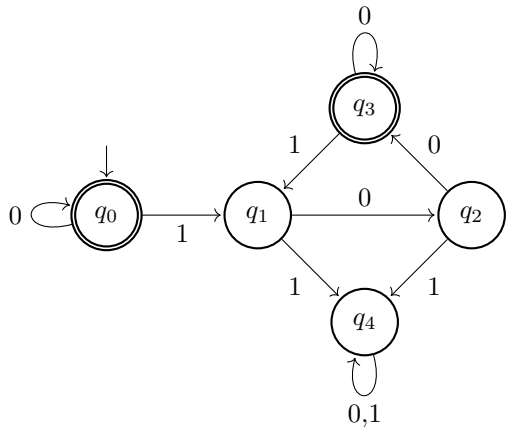
(*ii*) Let $L = L(M)$ for a DFA $M = (Q,\Sigma,\delta,q_0,F)$.

Intuitively, reversing all of the arrows in the state diagram for M and swapping the initial and accepting states should yield a state diagram that accepts the reverse of every word in L. However, M may have multiple accept states, so this naïve inversion doesn't yield a valid DFA (or NFA). However, we may amend this by adjoining a new initial state q_I with ε-transitions to the intended initial states.

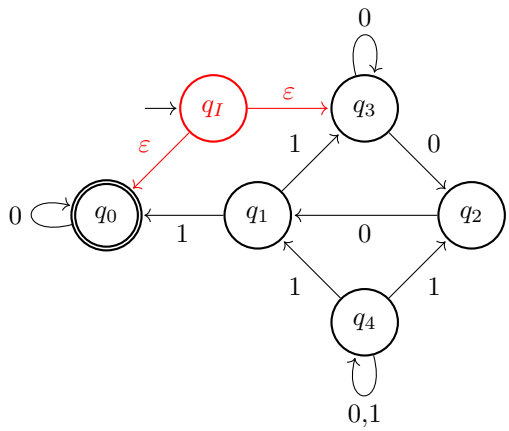
Construct the NFA $M^r = \big(Q \cup \{q_I\},\Sigma,\delta^r,q_I,\{q_0\}\big)$ by

$$\begin{aligned}
\delta^r(q_I,\varepsilon) &\coloneqq F \\
\delta^r(q_I,a) &\coloneqq \varnothing \text{ for all } a \in \Sigma \\
\delta^r(q,\varepsilon) &\coloneqq F \text{ for all } q \in Q \text{ (since we started with a DFA)} \\
\delta^r(q,a) &\coloneqq \big\{q' \in Q \mid \delta(q',a) = q\big\}
\end{aligned}$$

Example. Consider the language $L \subseteq \{0,1\}^*$ defined by $L = \{\text{every 1 is followed by 00}\}$, accepted by the DFA



By reversing the arrows and adding a new state equipped with ε-transitions to the two previous accepting states, we obtain a state diagram of an NFA that accepts $L^r = \{\text{every 1 is preceded by 00}\}$:



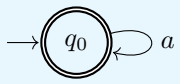
$\triangle$

2 Sheet 2

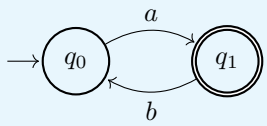
$Q1$

Construct a DFA equivalent to each of the NFA below, where $\Sigma = \{a,b\}$.

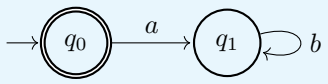
(i)



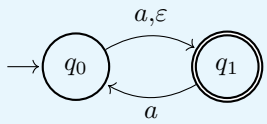
(ii)



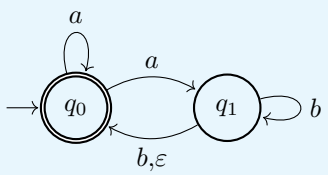
(iii)



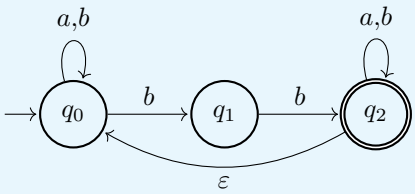
(iv)



(v)



(vi)



When a DFA is run on a word, we just keep track of a single state; that is, the state q_i that is reached upon reading a prefix of the input string, that can then be overwritten by the state that is reached upon reading the next symbol s.

In contrast, when running an NFA, we need to keep track of the set of all states that could be reached after seeing the same prefix, according to the non-deterministic choices made by the automaton. If, however, after a certain prefix has been read, a set S of states can be reached, then the set of symbols

reachable upon reading the next symbol s is a deterministic function of S and s. That is, while the states reached at each step in an individual run is non-deterministic, the set of states reachable at each step over all possible runs is fully deterministic, and as such, traversing sets of reachable states in this way describes the action of a DFA. This is the strategy of our construction.

Let $N = (Q,\Sigma,\delta,q_0,F)$ be the NFA to be determinised. Then, the DFA $M = (Q',\Sigma,\delta',q_0',F')$ defined by

- $Q' = \mathcal{P}(Q)$;
- $q_0' = \textsc{eclose}(q_0)$;
- $F' = \{X \subseteq Q : X \cap F \neq \emptyset\}$;
- $\delta'(X,a) = \bigcup_{x \in X} \textsc{eclose}\big(\delta(x,a)\big)$
  $= \Big\{z : \exists x \in X : z \in \textsc{eclose}\big(\delta(x,a)\big)\Big\}$

accepts the same language.

Proposition. For all $s \in \Sigma^*$, $\hat{\delta}(q_0,s) = \hat{\delta}'(q_0',s)$.

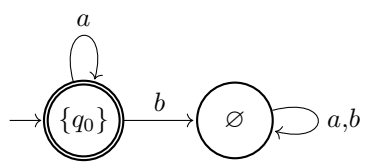
Proof. By induction on $|s|$. If $|s| = 0$, then $\hat{\delta}(q_0,s) = \hat{\delta}(q_0,\varepsilon) = \textsc{eclose}(q_0) = q_0' = \delta'(q_0',\varepsilon) = \hat{\delta}'(q_0',s)$. Otherwise, suppose $s = wa$ with $w \in \Sigma^*$, $a \in \Sigma$. Then,

$$\begin{aligned}
\hat{\delta}(q_0,s) &= \bigcup_{p \in \hat{\delta}(q_0,w)} \textsc{eclose}\big(\delta(p,a)\big) \\
&= \bigcup_{p \in \hat{\delta}'(q_0',w)} \textsc{eclose}\big(\delta(p,a)\big) \\
&= \delta'\big(\hat{\delta}(q_0',w),a\big) \\
&= \hat{\delta}(q_0',s)
\end{aligned}$$

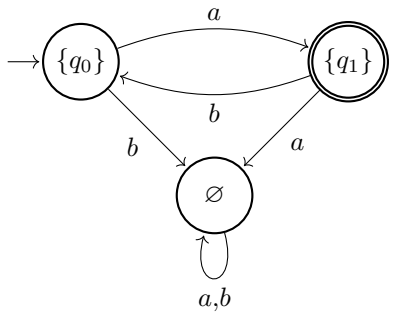
■

In this question, there is only one accepting state for each NFA, so we may omit adjoining a new initial state and simply use the accepting state directly.

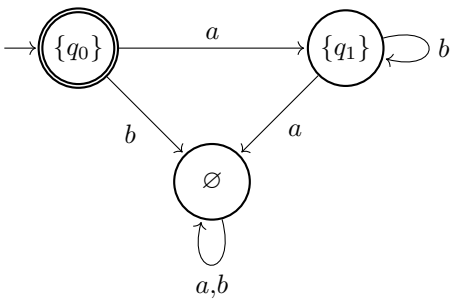
(*i*)



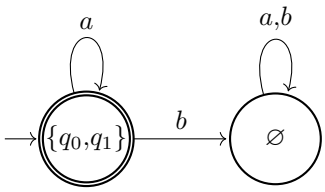
(*ii*)



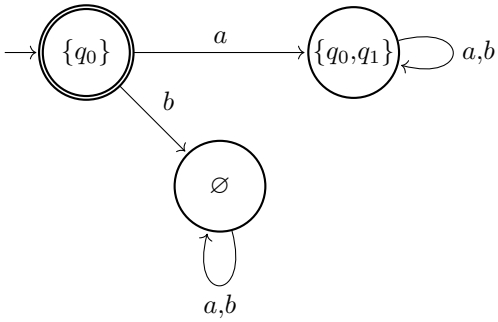
(*iii*)



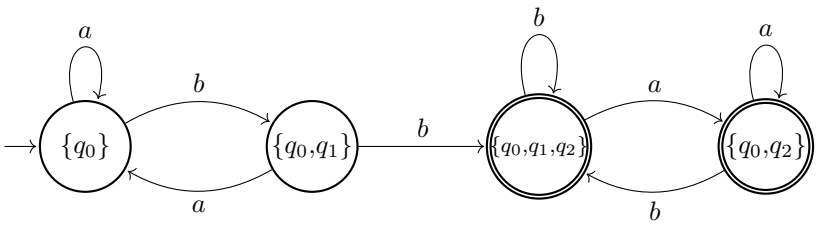
(iv)



(v)



(vi)



$Q2$

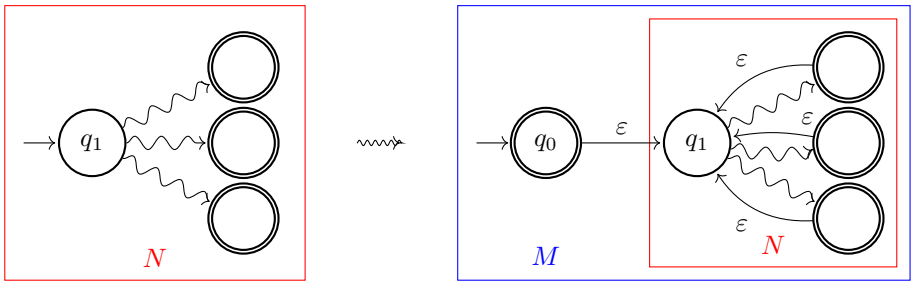
Consider the language $L = (01 \cup 001 \cup 010)^*$.

(i) Construct an NFA that recognises L.

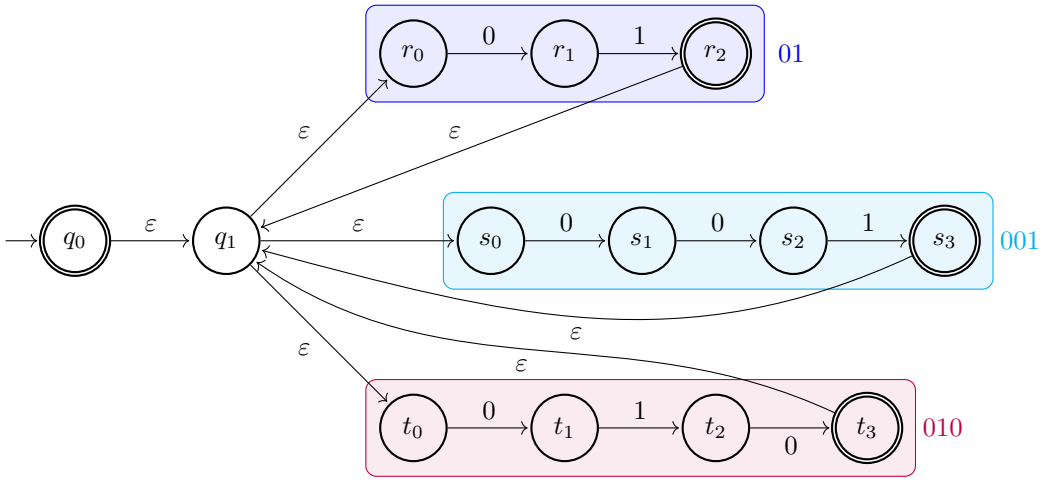
(ii) Determinise this NFA.

If L is recognised by N, then L^* is recognised by the NFA $M = \big(Q_1 \cup \{q_0\}, \Sigma, \delta, q_1, F_1,\big)$ formed by adding a new starting state q_0 that is also accepting (in order to accept the empty string) with an ε-transition to

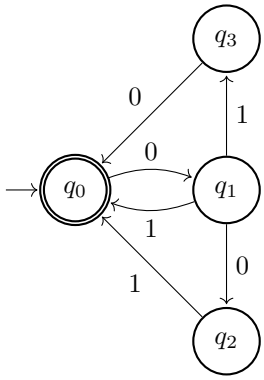
the previous start state, then adding ε-transitions from the previous accepting states to the start state (to allow for arbitrary concatenations of words accepted by N):



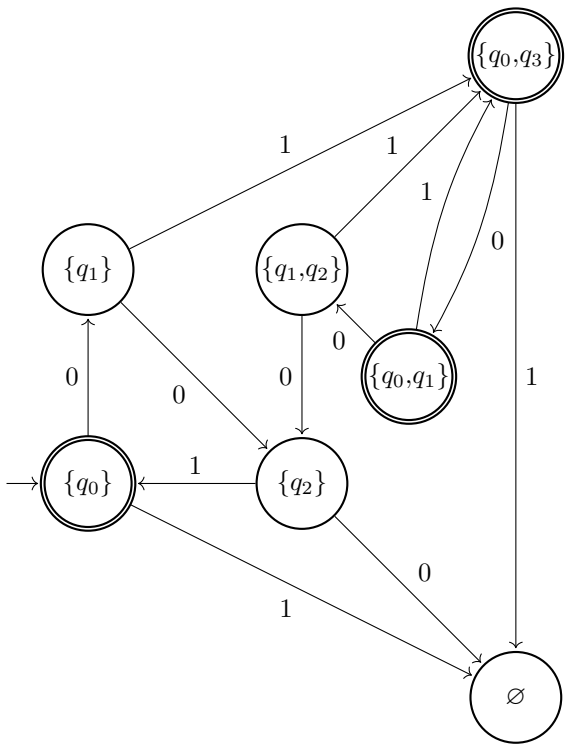
(i)



We can simplify this to:



(ii) Determinising via subset construction:



*Q*3

Prove that the following languages are not regular:

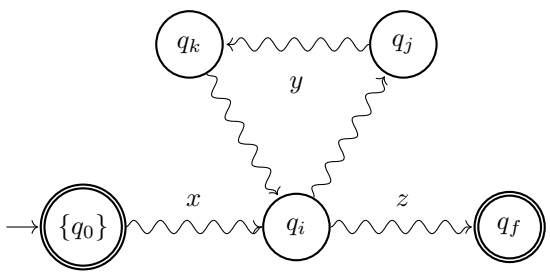
(*i*) $L_0 = \{a^j b^m \mid j > m \geq 0\}$;

(*ii*) $L_1 = \{a^i b^j c^k \mid i,j,k \geq 0 \text{ and if } i = 1 \text{ then } j = k\}$;

(*iii*) $L_2 = \{0^n 1^m 0^n \mid m,n \geq 0\}$;

(*iv*) $L_3 = \{0^m 1^n \mid m \neq n\}$.

Every regular language is recognised by a DFA – a deterministic *finite* automaton. Because a DFA only has finitely many states, any sufficiently long string s must repeat a state in the DFA, and hence pass through a cycle:



That is, s must be of the form xyz, where x is the string read before the cycle, y is the string given by the cycle itself, and z is the trailing string read before accepting.

Because the DFA has no memory, the cycle may be repeated any number of times with the same substring y still be accepted as long as the prefix x and suffix z remain the same. So this diagram accepts any

string starting with x, followed by any number of ys (including zero), followed by z.

In fact, this is a complete characterisation of the DFAs that accept an infinite language: because DFAs must have finitely many states, this is the only way an infinite regular language can arise.

Lemma (Pumping Lemma for Regular Languages)**.** *Let L be a regular language. Then, there exists an integer $p \geq 1$ (the pumping length), such that for every string $s \in L$ with length $|s| \geq p$, there exists a decomposition $s = x \cdot y \cdot z$ such that*

- $|y| \geq 1$;
- $|xy| \leq p$;
- *for all $n \geq 0$, $x \cdot y^n \cdot z \in L$.*

The lemma is just a formalisation of these observations. It says that for every regular language, every word longer than a certain length must be of this form where we can repeat some middle segment of the word and still be accepted.

We can also interpret the pumping length p as the number of internal states of the (minimal) DFA that recognises the language, so if a word is longer than p, it must revisit a state, and hence we have a state diagram of the shape above.

(*i*) Suppose L_0 is regular and let p be its pumping length. Consider the string $s = a^{p+1}b^p$ of length $2p + 1 > p$ and let $s = xyz$ be a decomposition with $|y| \geq 1$ and $|xy| \leq p$.

Since $|xy| \leq p$, the substring xy must consist entirely of a's since the first $p + 1$ symbols of the string are all a's. By the pumping lemma, $xy^nz \in L$ for all $n \geq 0$, but pumping y zero times reduces the number of a's, yielding a string not in L_0. So L_0 does not satisfy the pumping lemma for regular languages and is hence not a regular language.

(*ii*) Recall that regular languages are closed under intersection: by the contrapositive, if $L = U \cap V$ is non-regular and U is regular, then V is non-regular. So, to show that L_1 is non-regular, it suffices to show that $L_1 \cap U$ is non-regular for some regular language U.

Consider the regular language U given by the regular expression $a(b + c)^*$. Then,

$$L \coloneqq L_1 \cap U = \{ab^jc^j \mid j \geq 0\}$$

Suppose L is regular and let p be its pumping lenth. Consider the string $s = ab^pc^p$, and let $s = xyz$ be a decomposition with $|y| \geq 1$ and $|xy| \leq p$.

Since $|xy| \leq p$, $xy = ab^m$ for some $m < p$. If $x = \varepsilon$, then $y = ab^m$ and pumping y zero times removes the leading a. Otherwise, if $x \neq \varepsilon$, then $y = b^n$ for some $n < p$. Now, pumping y changes the number of bs but not the number of cs. So no such decomposition exists and L is non-regular, and hence L_1 is non-regular.

(*iii*) Suppose L_2 is regular and let p be its pumping length. Consider the string $s = 0^p1^{p+1}0^p$ and let $s = xyz$ be a decomposition satisfying the hypotheses of the pumping lemma. Since $|xy| \leq p$, y consists entirely of 0s, so pumping y changes the number of 0s in the initial segment, which leaves the language.

(*iv*) Regular languages are closed under complementation, so it suffices to show that $\overline{L_3} = \{0^n1^n\}$ is non-regular. Suppose $\overline{L_3}$ is regular and let p be its pumping length. Consider the string $s = 0^p1^p$ with a decomposition $s = xyz$ satisfying the hypotheses of the pumping lemma. Since $|xy| \leq p$, y consists entirely of 0s, so pumping y changes the number of 0s in the initial segment, which leaves the language.

Q4

Consider the following language over the alphabet $\Sigma = \{a,b\}$:

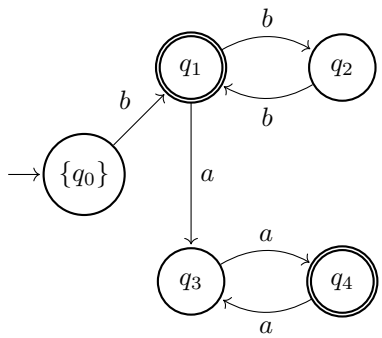
$$L = \left\{ w \;\middle|\; \begin{array}{c} w \text{ has an even number of } a\text{'s,} \\ \text{an odd number of } b\text{'s,} \\ \text{and does not contain the substring } ab \end{array} \right\}$$

(*i*) Prove that L is regular.

(*ii*) Find a regular expression that generates L.

First note that 0 is not an odd number, so w contains at least one b; and since w cannot contain the substring ab, all of the a's must appear after the b(s). So, equivalently, L has the simpler presentation $L = \{b^{2n+1}a^{2n}\}$.

(*i*) To show that L is regular it suffices to provide an NFA recognising L:

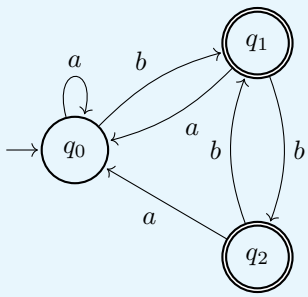


(*ii*) Using $L = \{b^{2n+1}a^{2n}\}$, this becomes simple:

$$b(bb)^*(aa)^*$$

Q4

Consider the following DFA:



(*i*) Show that q_1 and q_2 are bisimilar by giving a bisimulation $R \ni \langle q_1,q_2 \rangle$.

(*ii*) Compute a regular expression equivalent to the automaton.

(*iii*) Prove the following result:

Theorem. *States that are bisimilar are equivalent.*

Recall that a bisimulation on a DFA $M = (Q,\Sigma,\delta,q_0,F)$ is a relation $R \subseteq Q \times Q$ on states such that, for any $\langle q,q' \rangle \in R$:

- Acceptance agreement: $q \in F$ if and only if $q' \in F$;
- Step correspondence: for every $a \in \Sigma$, $\big\langle \delta(q,a),\delta(q',a) \big\rangle \in R$.

That is, two states are bisimilar if they agree on ε, and any single step from the two states yields bisimilar states.

While not strictly required by the definition, it is often sensible to add the diagonal elements $\langle q_i,q_i \rangle$ to R (the *reflexive closure*) since the conditions are trivial for such pairs, and similarly, if $\langle q_i,q_j \rangle \in R$, it can be helpful to add the reversed pair $\langle q_j,q_i \rangle$ to R (the *symmetric closure*) since the conditions are symmetric in q_i and q_j.

(i) We require $\langle q_1,q_2 \rangle \in R$. Taking the symmetric reflexive closure, we have:

$$R = \big\{ \langle q_0,q_0 \rangle, \langle q_1,q_1 \rangle, \langle q_2,q_2 \rangle, \langle q_1,q_2 \rangle \langle q_2,q_1 \rangle \big\}$$

The two conditions are trivial for all of the reflexive pairs; and we only need to check one of $\langle q_1,q_2 \rangle$ and $\langle q_2,q_1 \rangle$.

We verify the two conditions for $\langle q_1,q_2 \rangle \in R$:

- Acceptance agreement: $q_1 \in F$ and $q_2 \in F$;
- Step correspondence:

$$\big\langle \delta(q_1,a),\delta(q_2,a) \big\rangle = \langle q_0,q_0 \rangle \in R$$
$$\big\langle \delta(q_1,b),\delta(q_2,b) \big\rangle = \langle q_2,q_1 \rangle \in R$$

so R is a bisimulation.

Recall that a GNFA is a 5-tuple $(Q,\Sigma,q_{\text{start}},q_{\text{accept}},\delta)$, consisting of

- a finite set Q of states;
- a finite alphabet Σ;
- the start state, $q_{\text{start}} \in Q$;
- the accept state, $q_{\text{accept}} \in Q$;
- and a transition function $\delta : \big(Q \setminus \{q_{\text{accept}}\}\big) \times \big(Q \setminus \{q_{\text{start}}\}\big) \to \mathcal{R}$, where $\mathcal{R}$ is the set of all regular expressions over Σ.

Intuitively, a GNFA is an NFA with at most one transition (technically, exactly one, but we may omit the edge in state diagrams when the expression is $\emptyset$) between any two states, but labelled with a regular expression instead of just a single alphabet symbol.

We can convert a DFA into a regular expression as follows:

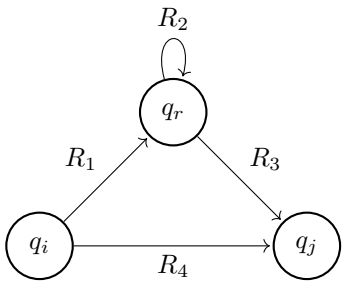
We first transform the DFA into a GNFA as follows:

1. Add a new start state with an ε-transition to the previous start state.

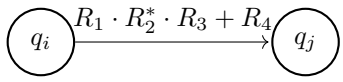
2. Add a new accepting state with ε-transitions from the previous accepting states to this state.
3. Any transitions with multiple labels may be replaced with a transition labelled with the union of the previous labels.

This GNFA may then be converted into a regular expression as follows:

1. If there are only two states, then we are done, as these must be the unique start and accepting states, and transition connecting them is a regular expression.
2. Otherwise, select some state $q_r \in Q \setminus \{q_{\text{start}}, q_{\text{end}}\}$. Then, simultaneously for all $(q_i, q_j) \in \big(Q \setminus \{q_{\text{start}}, q_r\}\big) \times \big(Q \setminus \{q_{\text{end}}, q_r\}\big)$, we replace the transitions

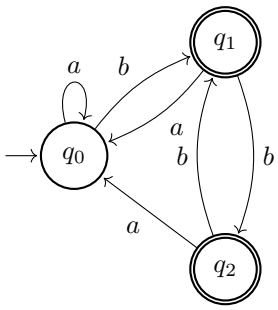


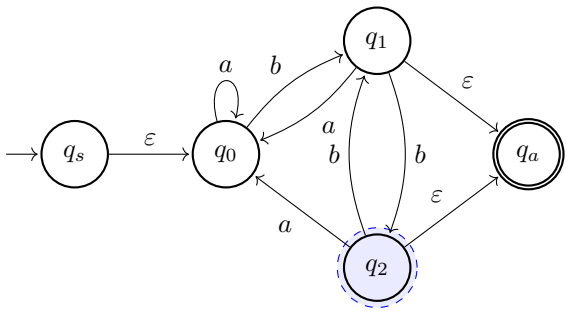
by the single edge

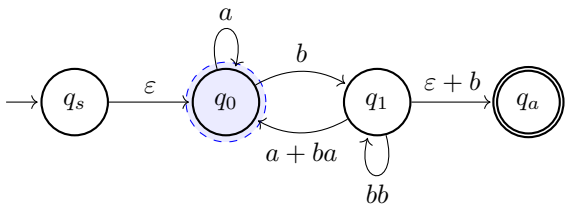


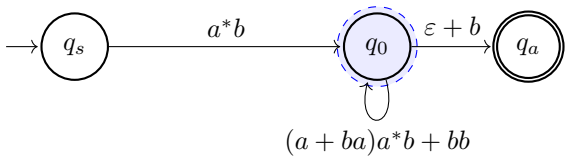
Once this has been done, we may remove q_r from the diagram, then pick a new state to be q_r, until the diagram has only the initial and accepting state remaining.

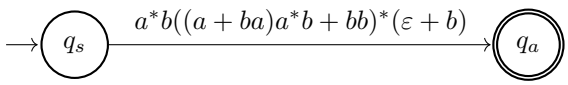
(ii) The state deleted at each stage is highlighted in blue.





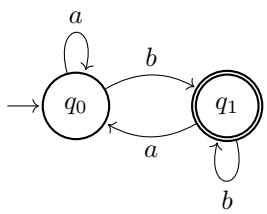


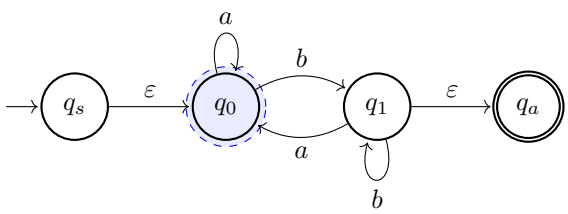


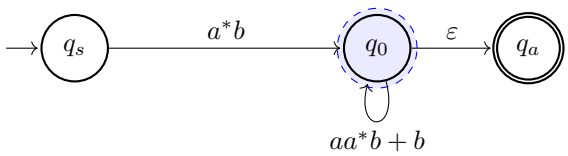


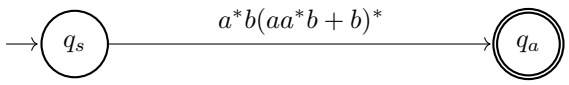
So a regular expression equivalent to the original DFA is $a^*b((a + ba)a^*b + bb)^*(\varepsilon + b)$.

We could have also identified the bisimilar states to obtain a simpler DFA before converting it into a GNFA:









So another regular expression equivalent to the original DFA is given by $a^*b(aa^*b+b)^*$.

Recall that two states q,q' of a DFA $M = (Q,\Sigma,\delta,q_0,F)$ are equivalent if the languages $L_q = L(M_q)$ and $L_{q'} = L(M_{q'})$ defined by the DFA $M_q = (Q,\Sigma,\delta,q,F)$ and $M_{q'} = (Q,\Sigma,q',\delta,F)$ that have initial states replaced by q and q' respectively, are equal.

(iii) By induction on $|s|$. Let q,q' be bisimilar.

For $s = \varepsilon$, $q \in F$ if and only if $q' \in F$ by acceptance agreement of bisimilar states, so $\varepsilon \in L_q$ if and only if $\varepsilon \in L_{q'}$.

Otherwise, suppose $s = aw$ with $w \in \Sigma^*$ and $a \in \Sigma$. By step correspondence, $\langle \delta(q,a),\delta(q',a) \rangle \in R$, and by the induction hypothesis, $w \in L_{\delta(q,a)}$ if and only if $w \in L_{\delta(q',a)}$, so $aw \in L_q$ if and only if $aw \in L_{q'}$.

3 Sheet 3

$Q1$

Give context-free grammars that generate the following languages over the alphabet $\Sigma = \{0,1\}$:

(i) $L_1 = \{w \mid w \text{ contains at least three occurences of } 1\}$;

(ii) $L_2 = \{w \mid w \text{ starts and ends with the same symbol}\}$;

(iii) $L_3 = \{w \mid |w| \text{ is odd and its middle symbol is } 0\}$;

(iv) $L_4 = \varnothing$;

(v) $L_5 = \{\varepsilon\}$.

(i) We need three occurences of 1, and anything can go between or around them, so we can just immediately insert $A1A1A1A$, where A generates $(0+1)^*$:

$$S \longrightarrow A1A1A1A$$
$$A \longrightarrow AA \mid 0 \mid 1 \mid \varepsilon$$

The production for A can also be written linearly:

$$A \longrightarrow 0A \mid 1A \mid \varepsilon$$

(ii) We proceed similarly, immediately inserting the required matching pair of symbols (or taking care of edge cases like singletons and the empty word), and allow arbitrary expansion on the inside:

$$S \longrightarrow 0A0 \mid 1A1 \mid 0 \mid 1 \mid \varepsilon$$
$$A \longrightarrow AA \mid 0 \mid 1 \mid \varepsilon$$

(iii)

$$S \longrightarrow ASA \mid 0$$
$$A \longrightarrow 0 \mid 1$$

(iv) Any set of productions with no terminals will work:

$$S \longrightarrow S$$

(v)

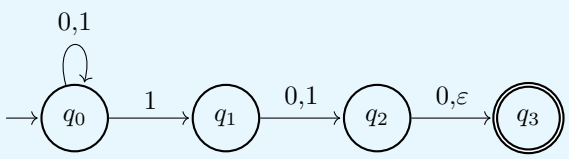
$$S \longrightarrow \varepsilon$$

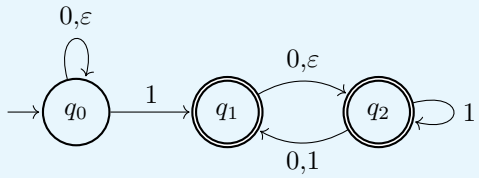
$Q2$

Give context-free grammars equivalent to the following NFA:

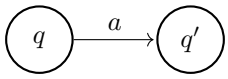
(i)

(*ii*)





We can convert NFA to CFGs by interpreting states as non-terminals, and transitions as right-linear production rules. That is, taking a transition



corresponds to adding a new terminal symbol a to the left and replacing the non-terminal Q with a new non-terminal Q'

$$Q \longrightarrow aQ'$$

and accepting states q_a correspond to productions of the shape

$$Q_a \longrightarrow \varepsilon$$

(*i*)

$$\begin{aligned} S &\longrightarrow Q_0 \\ Q_0 &\longrightarrow 0Q_0 \mid 1Q_0 \mid 1Q_1 \\ Q_1 &\longrightarrow 0Q_2 \mid 1Q_2 \\ Q_2 &\longrightarrow 0Q_3 \mid Q_3 \\ Q_3 &\longrightarrow \varepsilon \end{aligned}$$

(*ii*)

$$\begin{aligned} S &\longrightarrow Q_0 \\ Q_0 &\longrightarrow 0Q_0 \mid Q_0 \mid 1Q_1 \\ Q_1 &\longrightarrow \varepsilon \mid 0Q_2 \\ Q_2 &\longrightarrow \varepsilon \mid 1Q_2 \mid 0Q_1 \mid 1Q_1 \end{aligned}$$

$Q3$

Give context-free grammars equivalent to the following NFA:

(*i*) 0^*10^*

(*ii*) $1 \cup 0^*\emptyset$

(*iii*) $(01^+)^+$

(*i*)

$$S \longrightarrow Z1Z$$
$$Z \longrightarrow \varepsilon \mid 0Z$$

(*ii*) The symbol $\emptyset$ matches nothing, so $0^*\emptyset = \emptyset$ and $1 \cup \emptyset = 1$, so:

$$S \longrightarrow 1$$

(*iii*) Recall that $a^+ = aa^*$. That is, a^+ matches one or more copies of a.

$$S \longrightarrow 0A \mid 0AS$$
$$A \longrightarrow 1 \mid 1A$$

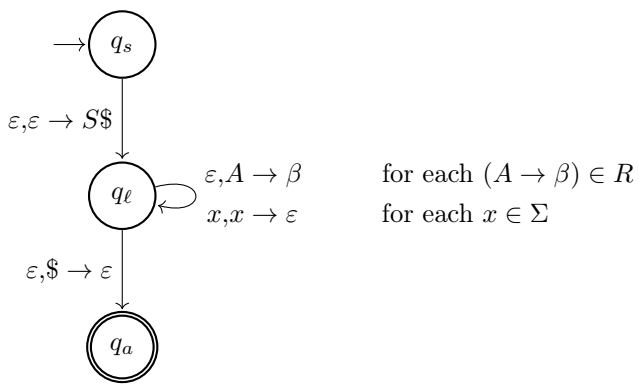
Q4

Provide an equivalent PDA for the language generated by the grammar $G = (V,\Sigma,R,E)$ with alphabet $\Sigma = \{a, (,), \times, +\}$, non-terminals $V = \{E,T,F\}$, and productions

$$R = \begin{Bmatrix} E \longrightarrow E + T \mid T \\ T \longrightarrow T \times F \mid F \\ F \longrightarrow (E) \mid a \end{Bmatrix}$$

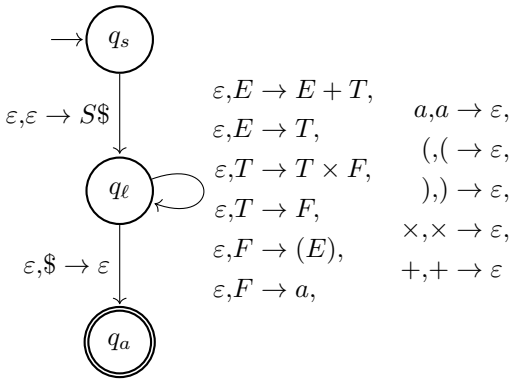
We can simulate a leftmost derivation of the grammar on a PDA by using the stack to represent the current sentential state.

1. First, push $S\$$ on to the stack to initialise the derivation at S, and using \$ for empty-stack detection, and move to a processing state.
2. For each production $A \to \beta$ in R, add an operation $\varepsilon,A \to \beta$, corresponding to (non-deterministically) applying a production rule for A.
3. For each terminal $a \in \Sigma$, add an operation $a,a \to \varepsilon$, allowing the PDA to match terminals produced by the grammar against the input string.
4. Once the top of the stack is \$, corresponding to a completed derivation, move to an accept state.

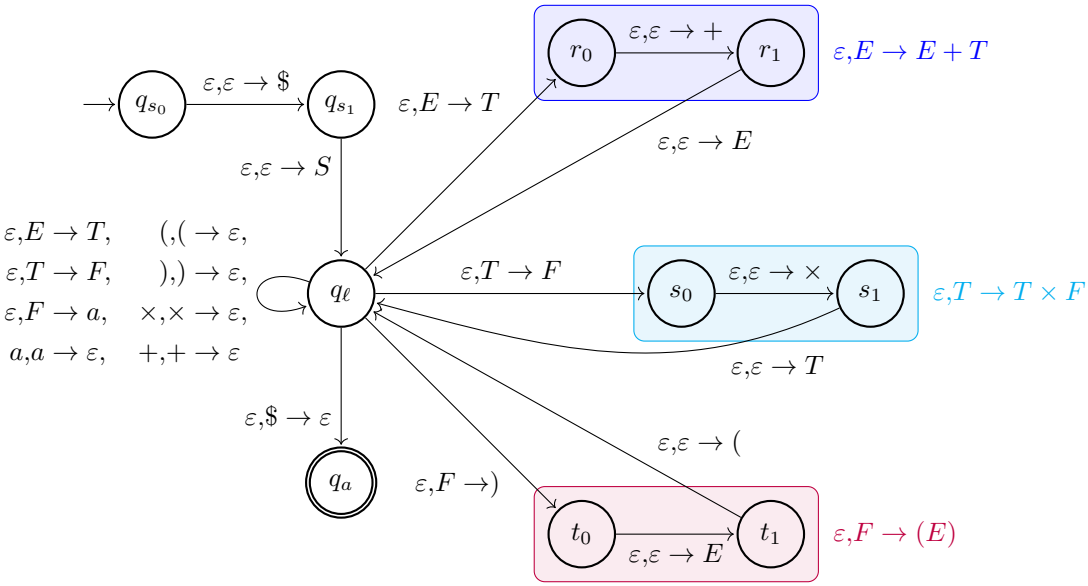


Note that we are implicitly using more than just 3 states when pushing multiple symbols to the stack: for the final PDA, any rules that produce multiple symbols can be expanded into multiple states that push the symbols on to the stack one at a time. Note that the symbols must be pushed on to the stack in reverse order, as stacks are first-in-last-out.

For this question, we have:



Expanding this out in full:



$Q5$

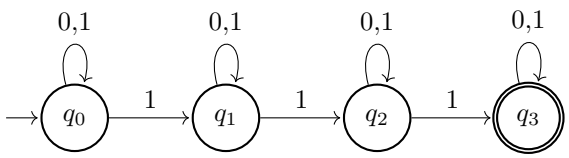
Construct a PDA for each of the following languages over the alphaber $\Sigma = \{0,1\}$:

(i) $L_1 = \{w \mid w \text{ contains at least three occurences of } 1\}$;

(ii) $L_2 = \{w \mid w \text{ starts and ends with the same symbol}\}$;

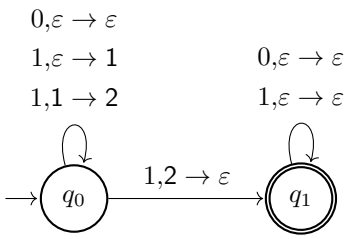
(iii) $L_3 = \{w \mid |w| \text{ is odd and its middle symbol is } 0\}$.

(i) This language is regular:

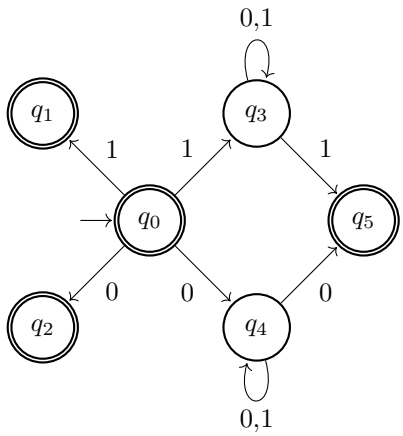


and every DFA is a PDA that happens to have all stack instructions $\varepsilon \to \varepsilon$.

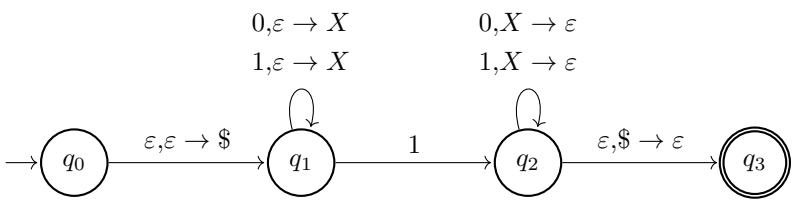
Alternatively, the stack can be used to count the number of 1's:



(*ii*) L_2 is also regular. An NFA accepting L_2 is as follows:

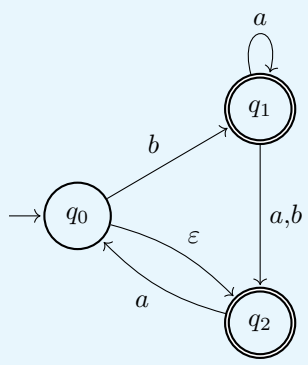


(*iii*) This language is not regular. The stack can again be used for counting: every time an input symbol is read, push the symbol X on to the stack and use non-determinism to find where the middle 0 is; then pop from the stack for each input symbol and accept on empty stack, ensuring there are the same number of symbols on each side of the 0.

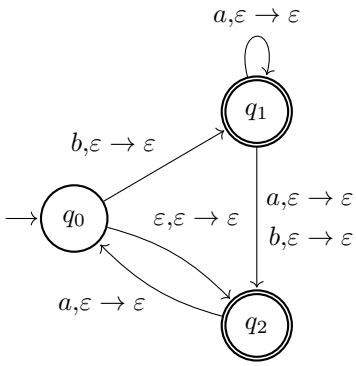


Q6

Construct an equivalent PDA for the following NFA:



Every NFA is a PDA that just doesn't use its stack.



Q7

Consider the following three languages over the alphabet $\Sigma = \{a,b,c\}$:

$$L_1 = \{a^i b^j c^k \mid i,j,k \geq 0, i = j = k\}$$
$$L_2 = \{a^i b^j c^k \mid i,j,k \geq 0, i = j \quad \}$$
$$L_3 = \{a^i b^j c^k \mid i,j,k \geq 0, \quad j = k\}$$

(*i*) Determine and prove which two are context-free.

(*ii*) Prove that the third language is not context-free.

(*iii*) Hence conclude that the class of context-free languages is not closed under intersection.

(*i*) Using a stack, we can compare two things for equality, so L_2 and L_3 are context-free. Context-free grammars that generate them are as follows:

$$L_2: \quad S \to XC \qquad C \to \varepsilon \mid cC \qquad X \to \varepsilon \mid aXb$$

$$L_3: \quad S \to AX \qquad A \to \varepsilon \mid aA \qquad X \to \varepsilon \mid bXc$$

Recall the pumping lemma for regular languages:

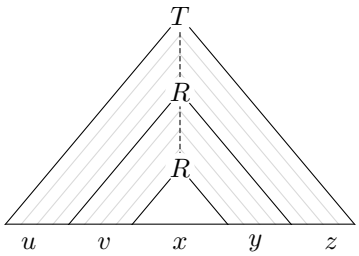
Lemma (Pumping Lemma)**.** *Let L be a regular language. Then, there exists an integer $p \geq 1$ (the pumping length), such that for every string $s \in L$ with length $|s| \geq p$, there exists a decomposition $s = x \cdot y \cdot z$ such that*

- $|y| \geq 1$;
- $|xy| \leq p$;
- *for all* $n \geq 0$, $x \cdot y^n \cdot z \in L$.

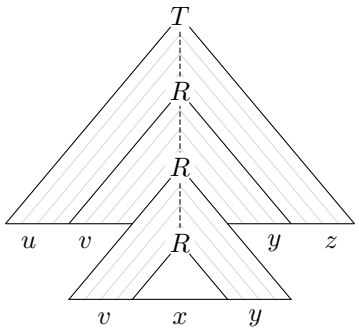
The intuition was that if an input string is too long, then it must loop somewhere inside the DFA.

A similar result holds for context-free languages, in that if a derived string is too long, it must repeat a variable somewhere in its parse tree. Similarly to before we can generate an infinite set of strings in the language by pumping this repeated variable:

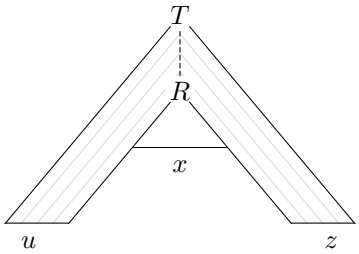
Suppose we repeat a variable R, and that the derivation between the first and second R yields a string that starts with a string v and ends with a string y.



Then, if we repeat R again, the derivation will repeat v and y:



We can also remove the repeated R:



Lemma 3.1 (Pumping Lemma)**.** *Let L be a context free language. Then, there exists an integer $p \geq 1$ (the pumping length) such that for every string $s \in L$ with length $|s| \geq p$, there exists a decomposition $s = u \cdot v \cdot x \cdot y \cdot z$ such that*

- $|vy| \geq 1$;

- $|vxy| \leq p$;
- *for all* $n \geq 0$, $u \cdot v^n \cdot x \cdot y^n \cdot z \in L$.

Proof. Let $G = (V,\Sigma,R,S)$ be a CFG, and suppose that the length of the longest string in the right side of a production rule in R is b. Then, any node in a parse tree yielding a string in $L(G)$ has at most b children. If the height of the tree is h, then it has at most b^h leaves, as each layer can only have b times as many nodes as the previous layer.

How long must a string be such that a variable is repeated on some root to leaf path of any parse tree of this string?

We claim that $p \coloneqq b^{|V|+1}$ is sufficient. Indeed, the parse tree of such a word must have $b^{|V|+1}$ leaves, so the tree must have height at least $|V| + 1$. But, there are only $|V|$ variables, so one must repeat.

Take the smallest parse tree for the string $w = uvxyz$. If $|vy| \not\geq 1$, then $v = y = \varepsilon$, and the section of the parse tree between the two Rs don't contribute anything, so $w = uxz$, and we have an even smaller parse tree for w, contradicting the minimality of the first tree.

If $|vxy| > p$, then the subtree rooted at the first R is itself long enough to have a repeated variable on a root to leaf path. So, we can split x into $x = u'x'y'$, so $w = (uv)v'x'y'(yz)$, which is still of the required form, but the middle substring has now decreased in length. We can then repeat this process until the middle substring has length at most p. ■

(*ii*) Suppose L_1 is context-free and p be its pumping length. Consider the string $s = a^p b^p c^p$ and let $s = uvxyz$ be a decomposition with $|vy| \geq 1$ and $|vxy| \leq p$. Since $|vy| \geq 1$, they can't both be empty, and since $|vxy| \leq p$, vxy contains at most 2 of the symbols. Pumping vxy zero times then reduces the number of only two of the symbols.

(*iii*) $L_1 = L_2 \cap L_3$; L_2 and L_3 are context-free, but L_1 is not. Note that this also proves that context-free languages are not closed under complementation since intersections can be constructed from unions and complements:

$$A \cap B = (A^c \cup B^c)^c$$

Q8

Consider the language

$$L_4 = \left\{u\#w \mid u,w \in \{0,1\}^*, u \text{ is a prefix of } w\right\}$$

(*i*) Find some examples of strings that belong to L_4 and some non-examples that do not.

(*ii*) Prove or disprove that L_4 is context-free.

(*i*) $\#,\#0,\#11,0\#01,11\#1100,\ldots \in L_4$ and $\varepsilon,01,0\#,1\#0,11\#1011,\ldots \notin L_4$.

(*ii*) Suppose that L_4 is context free and let p be its pumping length. Consider the string $s = 0^p1^p\#0^p1^p$ and let $s = uvxyz$ be a decomposition satisfying the hypotheses of the pumping lemma. Since $|vy| \geq 1$, they cannot both be empty.

- If v or y contains #, then pumping down removes the # from the string;
- If v and y are both to the left of #, then pumping upwards makes the left substring longer than the right substring, so it cannot be a prefix;
- If v and y are both to the right of #, then pumping downwards makes the right substring shorter than the left substring;
- Otherwise, v and y are on either side of the #. Since $|vxy| \leq p$, if v is non-empty, then it contains only 1's; and if y is non-empty it contains only 0's. Then, pumping upwards increases the number of 1's on the left and/or reduces the number of 0's on the right, so the left substring cannot be a prefix.

4 Sheet 4

$Q1$

(*i*) Can a Turing machine ever write the blank symbol $\sqcup$ on its tape?

(*ii*) Can the tape alphabet Γ be equal to the input alphabet Σ?

(*iii*) Can the head of a Turing machine ever stay on the same cell for two subsequent steps of a computation?

(*iv*) Can the state set of a Turing machine consist of only a single state?

(*i*) Yes: by definition, the tape alphabet Γ contains $\sqcup$, and the transition function $\delta : Q \times \Gamma \to Q \times \Gamma \times \{\mathsf{L},\mathsf{R}\}$ allows the machine to write any symbol from Γ to the tape, including $\sqcup$.

(*ii*) No: $\sqcup \in \Gamma \setminus \Sigma$.

(*iii*) Yes: if the move instruction is L while the head is on the leftmost cell, the head does not move.

(*iv*) No: q_{accept} and q_{reject} must be distinct.

$Q2$

Some of the following definitions are correct and some wrong (and some are even pure nonsense). Mark the correct definitions and for the incorrect ones underline the part of the definition which is wrong and explain why.

(*i*) A language L is recognisable if the language halts in the state q_{acc} whenever $x \in L$.

(*ii*) A language L is recognisable if there exists a Turing machine M such that M, given input x, halts in the state q_{accept} if and only if $x \in L$.

(*iii*) A language L is recognisable if it halts in an accepting state q_{accept} whenever $x \in L$.

(*iv*) A language L is recognisable if there exists a Turing machine M which has a state q_{accept} and is a member of L.

(*v*) A language L is recognisable if there exists a Turing machine M such that for any given input x, the run on x halts in the state q_{accept} if $x \in L$ and either loops or halts in q_{reject} if $x \notin L$.

(*vi*) A language L is recognisable if every Turing machine M, when run on a string x, halts in the state q_{accept} if and only if $x \in L$.

(*i*) Wrong: "A language L is recognisable if <u>the language halts</u> in the state q_{acc} whenever $x \in L$." – the language itself does not run/halt. Also, unbound variable x.

(*ii*) Correct.

(*iii*) Wrong: "A language L is recognisable if <u>it halts</u> in an accepting state q_{accept} whenever $x \in L$." – same as above.

(*iv*) Wrong: "A language L is recognisable if there exists a <u>Turing machine</u> M which has a state q_{accept} and <u>is a member of L</u>." – Turing machines are not (in general) elements of a language, so this doesn't make sense for a general definition.

(*v*) Correct.

(*vi*) Wrong: "A language L is recognisable if every Turing machine M, when run on a string x, halts in the state q_{accept} if and only if $x \in L$." – there only needs to exist a single Turing machine that halts on every string in L. (And since there exist Turing machines that recognise disjoint languages, no language satisfies this definition.)

.

Q3

Complete the definition below and be formally precise:

A language $L \subseteq \Sigma^*$ is decidable if and only if...

There are many equivalent definitions. Here are a few:

- A language $L \subseteq \Sigma^*$ is decidable if and only if there exists a Turing machine M such that M halts on a string x in q_{accept} if and only if $x \in L$ and halts in q_{reject} if and only if $x \notin L$.
- A language $L \subseteq \Sigma^*$ is decidable if and only if L is recognisable and $\Sigma^* \setminus L$ is recognisable.*
- A language $L \subseteq \Sigma^*$ is decidable if and only if there exists two Turing machines; one that enumerates exactly the strings in L in some order, and one that enumerates the strings in $\Sigma^* \setminus L$ in some order.

Q4

Consider the language

$$L = \{a^n b^n a^n \mid n \geq 0\}$$

(*i*) Prove that L is not context-free.

(*ii*) Draw a state diagram of a Turing machine that recognises L.

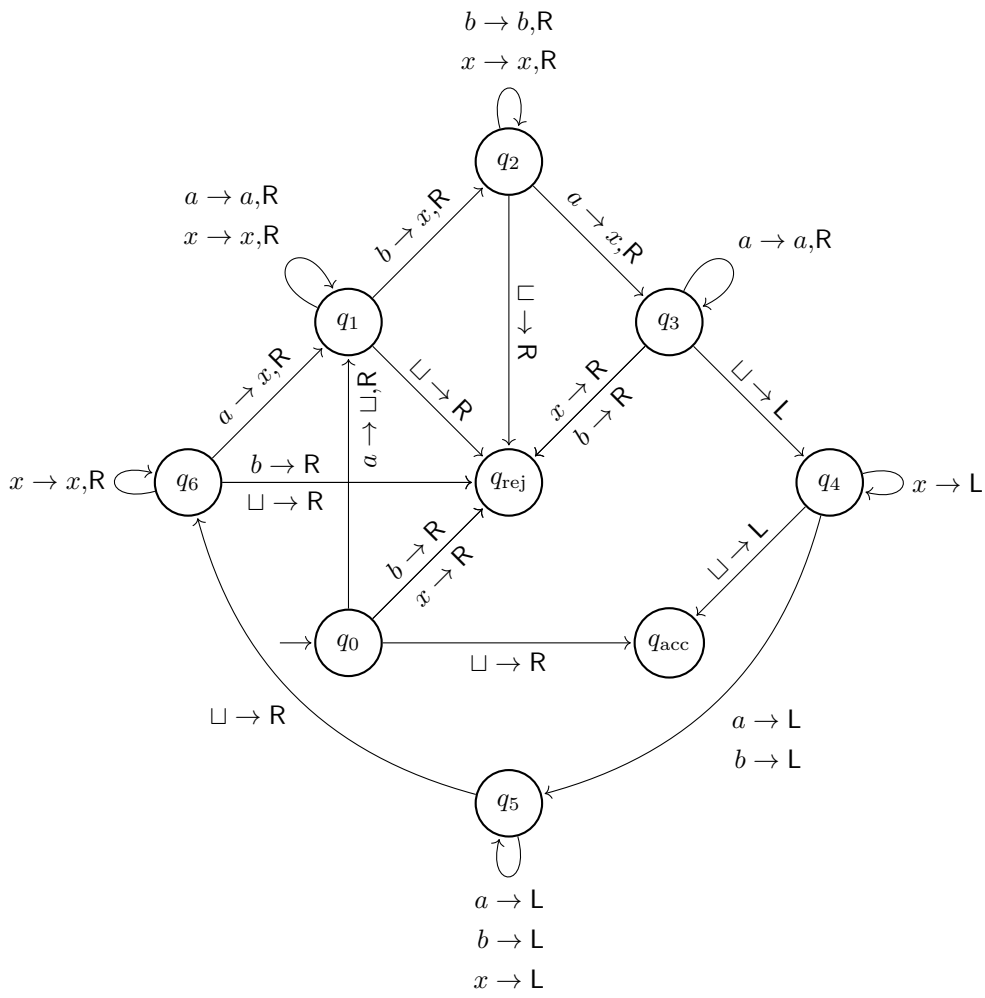
(*iii*) Consider the input string $s = aabbaa$. Trace the sequence of configurations that M enters when run on w.

(*iv*) Does M accept w?

(*i*) Suppose L is context-free and let p be its pumping length. Consider the string $s = a^p b^p a^p$ and let $s = uvxyz$ be a decomposition satisfying the hypotheses of the pumping lemma. Since $|vxy| \leq p$, vxy is either contained in first two parts $a^p b^p$ or the last two parts $b^p a^p$. In either case, pumping the string only changes the number of symbols in two of the three parts.

(*ii*)

*The class of Turing-decidable languages is also called the class of *recursive* languages, denoted by R; and the class of Turing-recognisable languages is also called the class of *recursively enumerable* languages, denoted by RE. This definition can then be phrased as R = RE∩co-RE, where co-RE is the class of languages whose complements are recursively enumerable. These names stem from recursion theory and mathematical logic, which studies a different, but equivalent, model of computation known as *general recursive functions*.



(iii)

$$q_0aabbaa$$
$$\sqcup q_1abbaa$$
$$\sqcup aq_1bbaa$$
$$\sqcup axq_2baa$$
$$\sqcup axbq_2aa$$
$$\sqcup axbxq_3a$$
$$\sqcup axbxaq_3\sqcup$$
$$\sqcup axbxq_4a$$
$$\sqcup axbq_5xa$$
$$\sqcup axq_5bxa$$
$$\sqcup aq_5xbxa$$
$$\sqcup q_5axbxa$$
$$q_5\sqcup axbxa$$
$$\sqcup q_6axbxa$$
$$\sqcup xq_1xbxa$$

$$\sqcup xxq_1bxa$$
$$\sqcup xxxq_2xa$$
$$\sqcup xxxxq_2a$$
$$\sqcup xxxxxq_3\sqcup$$
$$\sqcup xxxxq_4x$$
$$\sqcup xxxq_4xx$$
$$\sqcup xxq_4xxx$$
$$\sqcup xq_4xxxx$$
$$\sqcup q_4xxxxx$$
$$q_4\sqcup xxxxx$$
$$q_{\text{acc}}\sqcup xxxxx$$

(*iv*) The machine halted in the accepting state, so M accepts w.

Q5

Give implementation-level descriptions of Turing machines that decide the following languages over the alphabet $\Sigma = \{0,1\}$.

(*i*) $\{w \mid m$ contains an equal number of 0's and 1's$\}$

(*ii*) $\{w \mid m$ contains twice as many 0's as 1's$\}$

(*iii*) $\{w \mid m$ does not contain twice as many 0's as 1's$\}$

(*i*) Scan the tape, alternating between crossing off 0's and 1's and returning to the start between each one. If at any point we can't find a 0, scan right until: a blank is found, and accept; a 1 is found, and reject.

(*ii*) Same as above, but cross off two 0's in each loop before returning to the start.

(*iii*) Swap the accept and reject states in the previous Turing machine. This works because these machines are all deciders for their languages.

Q6

Consider "jumpy" Turing machines that offer a third movement instruction RR corresponding to moving 2 positions to the right on the tape in one step. Prove that jumpy Turing machines are no more expressive than ordinary ones.

First, note that every Turing machine is trivially a jumpy Turing machine that just doesn't use the RR instruction. Conversely, a jumpy Turing machine $M = (Q,\Sigma,\Gamma,\delta,q_0,q_{\text{accept}},q_{\text{reject}})$ can be simulated by an ordinary Turing machine as follows:

Add a new state q_{jump} to the Turing machine. For each jumpy transition $q \xrightarrow{a\to b,\mathsf{RR}} q'$, replace it by new transitions

- $q \xrightarrow{a\to b,\mathsf{R}} q_{\text{jump}}$;

- for each symbol $x \in \Gamma$, $q_{\text{jump}} \xrightarrow{x \to x, \mathsf{R}} q'$.

By replacing all jumpy transitions, this produces an ordinary Turing machine that simulates the original jumpy Turing machine.

5 Sheet 5

$Q1$

Is the language $\varnothing$ decidable?

Yes: it is recognised by the Turing machine that immediately goes into the reject state on start.

$Q2$

Is the following statement true?

If L is a decidable language, and $L' \subseteq L$, then L' is also decidable.

It is not: take $L = \Sigma^*$ and $L' = \text{HP}$ (or any other previously-known undecidable language).

$Q3$

Consider the following decision problem:

Given a regular expression R over $\Sigma = \{0,1\}$, does it describe a language that contains at least one string starting with 11 and ending with 0?

(i) Express this problem as a language RU_{110}.

(ii) Prove that RU_{110}. is decidable.

(i)

$$RU_{110} = \left\{ \langle R \rangle \;\middle|\; \begin{array}{c} R \text{ is a regular expression over } \{0,1\} \\ \text{and } \exists s \in \Sigma^*.\exists w \in L(R).w = 11s0 \end{array} \right\}$$

(ii) As usual, we will accept the Church–Turing thesis and only give a high-level algorithm for deciding this language:

- On input $\langle R \rangle$, convert R into an equivalent DFA M;
- From the start state, trace the run of M on the string 11 and mark the state s_0 that is reached;
- From each accepting state, trace a step backwards along a 0-transition and mark the set of states reached as T;
- Accept if there is a path from s_0 to some state in T, and reject otherwise. This is computable by any standard digraph reachability algorithm (one is described below).

Algorithm 1 Decide RU_{110}

```
procedure DECIDE(⟨R⟩)
    Convert R to an equivalent DFA M = (Q,Σ,q_0,F,δ)
    s_0 ← δ(δ(q_0,1),1)
    T ← {}
    for q ∈ Q do
        if δ(q,0) ∈ F then
            T.append(q)
        end if
    end for
    Visited ← {s_0}
    Stack ← [s_0]
    while Stack ≠ ∅ do
        q ← Stack.pop()
        if q ∈ T then
            return accept
        end if
        for a ∈ Σ do
            q′ ← δ(q,a)
            if q′ ∉ Visited then
                Visited.append(q′)
                Stack.append(q′)
            end if
        end for
    end while
    return reject
end procedure
```

Q4

Prove that the class of decidable languages is closed under:

(*i*) union;

(*ii*) and concatenation.

Let L_1 and L_2 be decidable languages, with deciders M_1 and M_2, respectively.

(*i*) We construct a decider for $L_1 \cup L_2$ as a 2-tape Turing machine as follows:

(1) On input x, copy x to the second tape;

(2) Simulate M_1 on the first tape and accept if M_1 accepts;

(3) Otherwise simulate M_2 on the second tape and accept if M_2 accepts, and reject otherwise.

Algorithm 2 Decide Union

```
procedure DECIDE(x)
    tape 2 ← x
    Simulate M_1 on tape 1
    if M_1 accepts then
        return accept
    else
        Simulate M_2 on tape 2
        if M_2 accepts then
            return accept
        else
            return reject
        end if
    end if
end procedure
```

(This is using short-circuit evaluation, but any valid disjunction evaluation strategy will work.) This defines a decider since M_1 and M_2 are deciders and hence the conditionals must terminate.

(*ii*) We construct a decider for $L_1 \circ L_2$ as a 3-tape Turing machine as follows:

(1) Input x;

(2) `For` $n = 0$ `to` $n = |x|$:

(a) Split x at the nth character into two words, i.e. into y and z with $|y| = n$ and $x = yz$, and copy y to the second tape and z to the third;

(b) Simulate M_1 on the second tape;

(c) If it rejects, `continue` (increment n and repeat);

(d) Otherwise, simulate M_2 on the third tape, and accept if it accepts;

(e) If it instead rejects, `continue`.

(a) Reject after all splits have been tested and rejected.

Algorithm 3 Decide concatenation

```
procedure DECIDE(x)
    for i = 0 to |x| do
        Split x at the nth character into y,z
        tape 2 ← y
        tape 3 ← z
        Simulate M_1 on tape 2
        if M_1 rejects then
            continue
        else
            Simulate M_2 on tape 3
            if M_2 accepts then
                return accept
            end if
        end if
    end for
    return reject
end procedure
```

Alternatively, we could also use non-determinism to split x rather than looping over all possible splits.

Q5

Which of the following languages are decidable?

(*i*) $L_1 = \{\langle A,B\rangle \mid A,B \text{ are DFA and } L(A) \cap L(B) = \varnothing\}$;

(*ii*) $L_2 = \{\langle M\rangle \mid M \text{ is a Turing machine and } M \text{ contains more than 5 states}\}$;

(*iii*) $L_3 = \{\langle M,w\rangle \mid M \text{ is a Turing machine and } M \text{ accepts } w \text{ in fewer than 1000 steps}\}$;

(*iv*) $L_4 = \{\langle M,w\rangle \mid M \text{ is a Turing machine and } M \text{ accepts } w \text{ in a finite number of steps}\}$.

(*i*) L_1 is decidable. On input $\langle A,B\rangle$, we can compute the DFA C that recognises $L(A) \cap L(B)$, and the non-emptiness of regular languages is decidable.

(*ii*) L_2 is decidable. On input $\langle M\rangle$, enumerate the states of M, accepting once 6 states are counted and rejecting if the count terminates before then.

(*iii*) L_3 is decidable. On input $\langle M,w\rangle$, simulate the run of M on w for 1000 steps, returning whatever M returns. If 1000 steps have been simulated and M has not terminated, reject.

(*iv*) $L_4 = \mathrm{A_{TM}}$ and is not decidable.

Q6

Consider the following decision problem:

Given a Turing machine M, are there exactly n distinct input strings that M will accept?

(*i*) Express this problem as a set of languages L_n parametrised by n.

(*ii*) Prove by a reduction that L_n is undecidable.

(*i*) $L_n = \{\langle M \rangle \mid M \text{ is a Turing machine and there exist exactly } n \text{ distinct strings that } M \text{ accepts}\}$

(*ii*) Let $S = \{s_1, \ldots, s_n\}$ be a set of n distinct strings. We give a decider for $\mathrm{A_{TM}}$.

(1) Input $\langle M,w \rangle$.

(2) Construct a new Turing machine M' as follows:

(a) Input x.

(b) If $x \in S$, accept.

(c) Otherwise, simulate M on w and accept if it accepts.

(3) Call the L_n oracle on M' and return the opposite.

If $\langle M,w \rangle \notin \mathrm{A_{TM}}$, then M does not accept w, so M' accepts exactly the n strings in S, and $\langle M' \rangle \in L_n$; and conversely, if $\langle M,w \rangle \in \mathrm{A_{TM}}$, then M accepts w, so M' accepts every string, and $M' \notin L_n$.

So $\langle M' \rangle \in L_n \iff \langle M,w \rangle \notin \mathrm{A_{TM}}$, and hence $\overline{\mathrm{A_{TM}}} \leq_m L_n$, so any oracle for L_n is also a decider for $\mathrm{A_{TM}}$.

Q7

Let S be a language over the alphabet Σ with $S \neq \Sigma^*$ and $S \neq \varnothing$. Prove that if L is any decidable language, then $L \leq_m S$.

Recall that a mapping reduction from a language A over Σ to a language B over Γ is a function $f : \Sigma^* \to \Gamma^*$ that is:

- computable;
- and faithful: $w \in A$ if and only if $f(w) \in B$;

and we write $A \leq_m B$ if such a function exists.

Since S is a proper subset of Σ^*, there exists a string $t \in \Sigma^* \setminus S$; and since S is non-empty, there exists a string $s \in S$. Define a function by:

$$f(x) \coloneqq \begin{cases} s & x \in L \\ t & x \notin L \end{cases}$$

This function is faithful by construction and is computable since L is decidable, and hence defines a mapping reduction from L to S.

Q8

Prove that if A is recognisable, then $A \leq_m \mathrm{A_{TM}}$.

Suppose A is recognised by M. Define a function $f : A \to \mathrm{A_{TM}}$ by

$$f(x) \coloneqq \langle M,x \rangle$$

This function is computable since it simply concatenates the code of M with the string x, and is faithful since $x \in A$ if and only if M accepts x if and only if $f(x) = \langle M,x \rangle \in \mathrm{A_{TM}}$.

$Q9$

Are the following problems decidable?

Determine whether a given Turing machine M:

(i) takes more than 847 steps on the input ε?

(i) takes more than 847 steps on the input ε?

(ii) takes more than 847 steps on *some* input?

(iii) takes more than 847 steps on *all* inputs?

(iv) ever moves the read/write head more than 847 tape cells away from the left end-marker on input ε?

(v) ever moves the read/write head more than 847 tape cells away from the left end-marker on *some* input?

(vi) ever moves the read/write head more than 847 tape cells away from the left end-marker on *all* inputs?

(i) Yes. Simulate the run of M on ε for 847 steps, or until M halts, whichever happens first. If M halts before 847 steps, reject. Otherwise accept. As the number of steps is bounded and fixed, this will always halt, so this describes a decider.

(ii) Yes. Simulate the run of M on all possible inputs of length 847 for 847 steps, or until M halts, whichever happens earlier. Accept if any string doesn't halt in the first 847 steps.

There are $\Sigma_{n=0}^{847}|\Sigma|^n = (|\Sigma|^{848}-1)/(|\Sigma|-1)$ such inputs, which is finite, so this program terminates. It is also sufficient to only check these inputs, since if M runs for 847 steps on a word w of length greater than 847, then M also runs for at least 847 steps when the input is only the first 847 symbols of w, since M can only possibly access the first 847 symbols of w, in 847 steps.

(iii) Yes. The program for the previous case also decides this problem, accepting if all strings don't halt in the first 847 steps.

(iv) Yes. If the read/write head never goes past the 847th cell, then there are only finitely-many possible configurations that M could be in, since a Turing machine only has finitely many states and tape symbols. Specifically, there are

$$C = 847 \cdot |Q| \cdot |\Gamma|^{847}$$

possible total configurations since there are 847 positions the head can be in; $|Q|$-many internal states; and $|\Gamma|^{847}$ possible tape configurations. Once this number of steps has been exceeded, then by the pigeonhole principle, we must have repeated a configuration.

So, simulate the run of M on ε for $C+1$ steps, or until it halts, whicever happens earlier. If M ever moves the read/write head past the 847th cell, accept M; otherwise reject.

(v) Yes. Enumerate all strings of length at most 847 and run the previous program on them, accepting if any string moves the read/write head past the 847th cell, and rejecting otherwise.

(vi) Yes. The previous program mutatis mutandis also works here.

6 Sheet 6

$Q1$

Which of the following statements are true?

(i) $3n^2 + 2n + 7 \in O(n^2)$;

(ii) $3^n \in 2^{O(n)}$;

(iii) $n \in o(2n)$.

Recall that:

- $f \in O(g)$ if there exists $c > 0$ such that for all n, $|f(n)| \leq cg(n)$
- $f \in o(g)$ if for all $c > 0$, there exists N such that for all $n > N$, $|f(n)| \leq cg(n)$.

These definitions are very similar to those of limits, so we can also characterise these asymptotic complexity classes using limits as follows.

Let $f,g : D \to \mathbb{R}$ be functions defined on an unbounded domain D (usually $D = \mathbb{N}$ or $D = \mathbb{R}_{\geq 0}$) and suppose that g is eventually positive. Consider the quantity

$$L = \lim_{n\to\infty} \frac{f(n)}{g(n)}$$

- If $L = 0$, then $f \in o(g)$ (in particular, $f \in O(g)$);
- If $L < \infty$, then $f \in O(g)$;
- If $L = \infty$, then $f \in \omega(g)$ (in particular, $f \in \Omega(g)$);
- If $L > 0$, then $f \in \Omega(g)$;
- If $L \in (0,\infty)$, then $f \in \Theta(g)$.

Note that these conditions are sufficient and not necessary (specifically, L may fail to converge but f and g may still satisfy an asymptotic relation). However, these still serve as an easy check to compute.

The exact conditions are as follows:

$$L = \limsup_{x\to\infty} \frac{f(x)}{g(x)} \qquad \text{and} \qquad \ell = \liminf_{x\to\infty} \frac{f(x)}{g(x)}$$

- $L = 0$ if and only if $f \in o(g)$;
- $L < \infty$ if and only if $f \in O(g)$;
- $\ell = \infty$ if and only if $f \in \omega(g)$;
- $\ell > 0$ if and only if $f \in \Omega(g)$;
- $\ell > 0$ and $L < \infty$ if and only if $f \in \Theta(g)$.

(i) True:

$$\begin{aligned} 3n^2 + 2n + 7 &\leq 3n^2 + 2n^2 + 7n^2 \\ &= 12n^2 \end{aligned}$$

so $c = 12$ suffices.

Alternatively, using the limit definition:

$$\begin{aligned}\lim_{n\to\infty} \frac{3n^2+2n+7}{n^2} &= \lim_{n\to\infty} \frac{3n^2}{n^2} + \frac{2n}{n^2} + \frac{7}{n^2} \\ &= \lim_{n\to\infty} 3 + \frac{2}{n} + \frac{7}{n^2} \\ &= 3 \\ &< \infty\end{aligned}$$

(ii) True. The bases of exponentials don't matter for big-O since they can always be transformed into a multiplicative constant in the exponent:

$$\begin{aligned}3^n &= 2^{\log_2(3^n)} \\ &= 2^{\log_2(3)n}\end{aligned}$$

so $c = \log_2(3)$ suffices.

Alternatively, using the limit definition:

$$\lim_{n\to\infty} \frac{3^n}{2^{cn}} = \lim_{n\to\infty} \left(\frac{3}{2^c}\right)^n$$

This geometric limit converges if and only if $\frac{3}{2^c} \leq 1$, or equivalently, $3 \leq 2^c$, so any $\log_2(3) \leq c$ suffices.

(iii) False. Choose $c = \frac{1}{4}$, and for any N, choose $n = N + 1$. Then:

$$\begin{aligned}|n| &> \frac{n}{2} \\ &= \frac{1}{4} \cdot 2n\end{aligned}$$

So the condition fails for this choice of c, and hence $n \notin o(2n)$. (Any $c < \frac{1}{2}$ works as a counterexample.)

Alternatively, using the limit definition:

$$\begin{aligned}\lim_{n\to\infty} \frac{n}{2n} &= \lim_{n\to\infty} \frac{1}{2} \\ &= \frac{1}{2} \\ &\neq 0\end{aligned}$$

Q2

Let M be a 5-tape Turing machine running in $O(n^3)$ time. What is the time complexity of a single-tape Turing machine simulating M?

$O(n^6)$.

Given a k-tape Turing machine M, we can simulate it on a single-tape Turing machine S as follows:

- Interlace the k-tapes together onto a single tape:

$$
M\left\{\begin{array}{|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|}
\hline
\vdash & a_1^1 & a_1^2 & a_1^3 & a_1^4 & a_1^5 & a_1^6 & a_1^7 & a_1^8 & a_1^9 & a_1^{10} & a_1^{11} & a_1^{12} & a_1^{13} & a_1^{14} & a_1^{15} & \cdots \\
\hline
\vdash & a_2^1 & a_2^2 & a_2^3 & a_2^4 & a_2^5 & a_2^6 & a_2^7 & a_2^8 & a_2^9 & a_2^{10} & a_2^{11} & a_2^{12} & a_2^{13} & a_2^{14} & a_2^{15} & \cdots \\
\hline
\vdash & a_3^1 & a_3^2 & a_3^3 & a_3^4 & a_3^5 & a_3^6 & a_3^7 & a_3^8 & a_3^9 & a_3^{10} & a_3^{11} & a_3^{12} & a_3^{13} & a_3^{14} & a_3^{15} & \cdots \\
\hline
\end{array}\right.
$$

$$\vdots$$

$$
\begin{array}{|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|}
\hline
\vdots & & & & & & & & & & & & & & & & & \\
\hline
\vdots & & & & & & & & & & & & & & & & & \\
\hline
\vdots & & & & & & & & & & & & & & & & & \\
\hline
\vdots & & & & & & & & & & & & & & & & & \\
\hline
\vdots & & & & & & & & & & & & & & & & & \\
\hline
\vdots & & & & & & & & & & & & & & & & & \\
\hline
\vdots & & & & & & & & & & & & & & & & & \\
\hline
\vdots & & & & & & & & & & & & & & & & & \\
\hline
\vdots & & & & & & & & & & & & & & & & & \\
\hline
\vdots & & & & & & & & & & & & & & & & & \\
\hline
\vdots & & & & & & & & & & & & & & & & & \\
\hline
\vdots & & & & & & & & & & & & & & & & & \\
\hline
\vdots & & & & & & & & & & & & & & & & & \\
\hline
\vdots & & & & & & & & & & & & & & & & & \\
\hline
\vdots & & & & & & & & & & & & & & & & & \\
\hline
\end{array}
$$

$$
S\ \begin{array}{|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|}
\hline
\vdash & a_1^1 & a_2^1 & a_3^1 & \cdots & a_k^1 & a_1^2 & a_2^2 & a_3^2 & \cdots & a_k^2 & a_1^3 & a_2^3 & a_3^3 & \cdots & a_k^3 & \cdots \\
\hline
\end{array}
$$

$$\underbrace{\qquad\qquad}_{\text{First cells}}\underbrace{\qquad\qquad}_{\text{Second cells}}\underbrace{\qquad\qquad}_{\text{Third cells}}$$

  The position of the head of each individual tape can then be recorded with an extended alphabet $\hat{\Gamma} = \Gamma \cup \dot{\Gamma}$, where a dotted symbol $\dot{a}$ represents the presence of a virtual head.

  The state space of S is then $Q \times \hat{\Gamma}^k \times \{\text{read,write}\}$, corresponding to tracking the state of M, the tuple of symbols currently read by the virtual heads, and a flag for reading and writing phases (see next step).

- Suppose M has run for n steps and the heads are at positions $p_1, \ldots, p_k \leq n$. The *active region* of S's tape then consists of the first $\max_i p_i$ blocks, which is of length at most $kn + k \in O(kn)$. S can then simulate the next step of M in two passes over the active region as follows:
  - Read: S scans across the active region, collecting the k dotted symbols $\dot{a}_i$ corresponding to the p_i positions of the virtual heads into its internal state.
  - Write: Using the collected tuple and the encoded current state of M, S can scan across the active region again, simulating the transitions of M on each block.

  After the second pass, S updates its internal encoding of M's state.

After n steps of M, each head has moved at most n cells, so the active region of S is $O(kn)$ in length, and each step of M takes $O(kn)$ steps of S. So, if M halts in $T(n)$ steps, then S halts in

$$
\begin{aligned}
\sum_{t=0}^{T(n)} O(kt) &= O(k) \sum_{t=1}^{T(n)} t \\
&= O\left(k \cdot \frac{T(n)\big(T(n)+1\big)}{2}\right) \\
&= O\big(k \cdot T(n)^2\big)
\end{aligned}
$$

Since the number of tapes k is a constant, this is $O\big(T(n)^2\big)$.

Q3

Let $\mathrm{ALL}_{\mathrm{DFA}} = \{\langle A\rangle \mid A \text{ is a DFA and } L(M) = \Sigma^*\}$. Show that $\mathrm{ALL}_{\mathrm{DFA}}$ is decidable.

Given an input $\langle A\rangle$, mark the initial state of A, and then repeatedly mark states reachable from currently marked states until no more transitions are available. If there exists at least one marked state that is not accepting, then reject. Otherwise accept.

Alternatively, compute $\langle \overline{A}\rangle$, where $L(\overline{A}) = \Sigma^* \setminus L(A)$. Since $\mathrm{E}_{\mathrm{DFA}}$ is decidable, calling its decider on $\langle \overline{A}\rangle$ decides $\mathrm{ALL}_{\mathrm{DFA}}$.

Q4

Let $\mathrm{E}^n_{\mathrm{CFG}} = \{\langle G\rangle \mid hg \text{ is a CFG and derives some word } x \text{ in at most } n \text{ steps.}\}$. Show that $\mathrm{E}^n_{\mathrm{CFG}}$ is decidable.

Given a CFG G, recursively define its *reachability stages* $\mathcal{R}_i$ as follows:

$$\mathcal{R}_0 = \{S\}$$
$$\mathcal{R}_{i+1} = \mathcal{R}_i \cup \{\beta \mid \exists \alpha \in \mathcal{R}_i . \alpha \Rightarrow_G \beta\}$$

That is, $\mathcal{R}_i$ contains the sentential states reachable in a most i productions.

To decide $\mathrm{E}^n_{\mathrm{CFG}}$, compute $\mathcal{R}_n$ and check if it contains any strings consisting only of terminal symbols. If it does, accept; otherwise reject.

This is computable since grammars can only have finitely-many productions, so the $\mathcal{R}_i$ are all finite and computable.

Q5

Consider the following decision problem:

$$\mathrm{Sub} = \{\langle N,x\rangle \mid N \text{ is an NFA and } x \text{ is a substring of some } y \in L(N)\}$$

Prove that Sub is decidable using the decidability of $\mathrm{E}_{\mathrm{DFA}}$.

x is a substring of y if and only if $\exists y_1,y_2 \in \Sigma^*. y_1xy_2 \in L(N)$, so define the language

$$\Sigma^* x \Sigma^* = \{y_1xy_2 \mid y_1,y_2 \in \Sigma^*\}$$

Then, $\langle N,x\rangle \in \mathrm{Sub}$ if and only if $L(N) \cap \Sigma^* x \Sigma^* \neq \varnothing$.

$\Sigma^* x \Sigma^*$ is regular as the concatenation of three regular languages and regularity is preserved under concatenation, so the non-emptiness of $L(N) \cap \Sigma^* x \Sigma^*$ is decidable as an instance of $\mathrm{E}_{\mathrm{DFA}}$.

Q6

Prove that the following language is decidable:

$$L_{\neg 000} = \left\{ \langle R \rangle \;\middle|\; \begin{array}{c} R \text{ is a regular expression over } \{0,1\} \text{ and} \\ L(R) \text{ does not contain any string with 000 as a substring} \end{array} \right\}$$

x contains 000 as a substring if and only if $\exists y_1,y_2 \in \Sigma^*.y_1 000 y_2 = x$. So $L(R)$ not containing any such strings is equivalent to $L(R) \cap \Sigma^* 000 \Sigma^* = \varnothing$, which is decidable as an instance of $\mathrm{E}_{\mathrm{DFA}}$.

Alternatively, convert R into a DFA A recognising the same language. Then, enumerate all 3-paths in A and check if any of them consist of three 0 transitions in a row. This is computable as a DFA is finite and hence there are only finitely many 3-paths to check.

Q7

Prove that the following language is undecidable:

$$\mathrm{EPSILON}_{\mathrm{TM}} = \left\{ \langle M \rangle \mid R \text{ is a Turing machine and } \varepsilon \in L(M) \right\}$$

We give a decider for A_{TM} assuming an $\mathrm{EPSILON}_{\mathrm{TM}}$ oracle.

(1) Input $\langle M,w \rangle$.

(2) Construct a new Turing machine M' as follows:

- Input x.
- Simulate M on w.

(3) Call the $\mathrm{EPSILON}_{\mathrm{TM}}$ oracle on M' and return what it returns.

If $\langle M,w \rangle \notin \mathrm{A}_{\mathrm{TM}}$, then M' either loops or rejects on every input and $M' \in \mathrm{EPSILON}_{\mathrm{TM}}$; conversely, if $\langle M,w \rangle \in \mathrm{A}_{\mathrm{TM}}$, then M' accepts every input and $M' \notin \mathrm{EPSILON}_{\mathrm{TM}}$.

So $\langle M' \rangle \in \mathrm{EPSILON}_{\mathrm{TM}} \iff \langle M,w \rangle \in \mathrm{A}_{\mathrm{TM}}$ and hence any oracle for $\mathrm{EPSILON}_{\mathrm{TM}}$ is also a decider for A_{TM}.

Q8

Is the following language decidable?

$$L = \left\{ \langle R_1,R_2 \rangle \mid R_1,R_2 \text{ are regular expressions and } L(R_1^*) = L(R_2) \right\}$$

Yes. Since regularity is preserved under Kleene star, $L(R_1^*) = L(R_2)$ is just checking equality of regular languages, which is decidable as follows:

Given regular languages A and B, construct their symmetric difference $C = A \triangle B = (A \setminus B) \cup (B \setminus A)$. Then $C = \varnothing$ if and only if $A = B$, and the emptiness problem for regular languages, i.e. $\mathrm{E}_{\mathrm{DFA}}$, is decidable.

Q9

Is the following language decidable?

$$L = \{\langle M \rangle \mid M \text{ accepts at least one } x \text{ in fewer than } |x| \text{ steps}\}$$

No. We give a decider for $\mathrm{A_{TM}}$ assuming an L oracle.

(1) Input $\langle M,w \rangle$.

(2) Construct a new Turing machine M' as follows:

- Input x.
- Simulate M on w.

(3) Call the L oracle on M' and return what it returns.

If $\langle M,w \rangle \notin \mathrm{A_{TM}}$, then M' rejects or loops on every input and $M' \notin L$.

Conversely, suppose $\langle M,w \rangle \in \mathrm{A_{TM}}$, with M accepting w in k steps, so on input x, M' also accepts x in k steps. If $k < |x|$, then we are done. Otherwise, let y be any word with length longer than k. Since M' ignores its input, it also accepts y in k steps, but $k < |y|$, so $M' \in L$.

So $\langle M' \rangle \in L \iff \langle M,w \rangle \in \mathrm{A_{TM}}$ and hence any oracle for L is also a decider for $\mathrm{A_{TM}}$.

7 Sheet 7

For this sheet, assume that all numbers are given in binary, and the adjacency relation on graphs are given by an adjacency matrix.

$Q1$

An (undirected) graph $G = (V,E)$ is *connected* if any pair of nodes are connected by a path. Show that

$$\text{CONNECTED} = \big\{\langle G\rangle \mid G \text{ is a connected graph}\big\}$$

is in P.

We give a decider with polynomial complexity in the number of vertices as follows:

(1) Input $\langle G\rangle$.

(2) Mark an node in G.

(3) `While` there is at least one unmarked node connected to a marked node, mark such an unmarked node.

(4) If all nodes are marked, accept. Otherwise reject.

Given $U \subseteq V$, we write $n(U)$ for the neighbourhood of U:

$$\begin{aligned} n(U) &\coloneqq \bigcup_{v\in X} \{w \mid \{w,v\} \in E\} \\ &= \big\{w \mid \exists v \in X.\{w,v\} \in E\big\} \end{aligned}$$

Algorithm 4 Decide graph connectivity

```
procedure DECIDE(⟨G⟩)
    Select a vertex v_0.
    marked ← {v_0}
    unmarked ← V(G) \ {v_0}
    while n(marked) ∩ unmarked ≠ ∅ do
        Select a vertex v ∈ n(marked) ∩ unmarked
        marked.append(v)
        unmarked.remove(v)
    end while
    if unmarked = ∅ then
        return accept
    else
        return reject
    end if
end procedure
```

The while-loop runs for at most $n \in O(n)$ steps, since we can only exhaust up to $|V|$-many vertices. Each loop requires checking for unmarked nodes connected to marked nodes, which takes $O(|E|) \subseteq O(n^2)$ time since we need to scan through (a subset of) the edges. The append and remove operations take $O(1)$ time if implemented as hash sets, bitsets, or boolean arrays; or $O(n)$ as dynamic arrays; or $O(\log n)$ as search trees, etc., but is polynomial in any case. Checking for emptiness is $O(1)$.

So overall, this naïve mplementation takes $O(n^3)$ time, so CONNECTED $\in$ P.

This can be optimised to $O(|V|+|E|) \subseteq O(n^2)$ time by maintaining a "frontier" of edges to check, rather than recomputing $n(\texttt{marked})$ every loop (as per BFS/DFS algorithms).

Q2

A *k-clique* is a complete induced subgraph of a graph. Show that

$$\textsf{3-CLIQUE} = \big\{\langle G\rangle \mid G \text{ is a graph with a 3-clique}\big\}$$

is in P.

There are $\binom{n}{3} = \frac{1}{6}n^3 - \frac{1}{2}n^2 + \frac{1}{3}n \in O(n^3)$ triples of vertices in G, and checking whether a given triple is a clique requires three $O(1)$ lookups (assuming an adjacency matrix representation of E). So checking if a 3-clique exists takes $O(n^3)$ time, so 3-CLIQUE

Q3

A *graph homomorphism* from a graph G to a graph H is a function $\varphi : G \to H$ such that $\big(\varphi(u),\varphi(v)\big) \in E(H)$ if $(u,v) \in E(G)$. That is, φ preserves the adjacency relation on vertices of G.

A graph homomorphism is an *isomorphism* if it has an inverse that is also a graph homomorphism, or equivalently, if it is bijective and preserves the adjacency relation in both directions. That is, $\big(\varphi(u),\varphi(v)\big) \in E(H)$ if and only if $(u,v) \in E(G)$. In this case, we say that G and H are isomorphic graphs. Informally, G and H are isomorphic if the nodes in G can be relabelled such that it becomes identical to H.

Show that

$$\textsf{GRAPH-ISO} = \big\{\langle G,H\rangle \mid G \text{ and } H \text{ are isomorphic graphs}\big\}$$

is in NP.

A certificate for this problem is a function $\varphi : G \to H$. Verifying that such a function is a graph isomorphism requires checking that it is a bijection and that it preserves the adjacency relation in both directions.

Checking injectivity requires looping over the vertices in G, storing their images, and checking that there are no duplicates, which is $O\big(|V(G)|\big)$ with a hash map, and checking surjectivity requires looping over the vertices of H and checking against the previous hash, which is $O\big(|V(G)|+|V(H)|\big)$.

Verifying the forward homomorphism property requires looping over the edges in G, which is $O\big(|V(G)|\big)$, and verifying that their image is in H, which is $O(1)$ assuming an adjacency matrix representation. The reverse homomorphism property is similar, taking $O\big(|V(H)|\big)$ time.

Overall, this takes $O\big(|V(G)|+|V(H)|+|E(G)|+|E(H)|\big)$ time, which is polynomial.

Q4

The *bin packing* problem is to determine, given a finite set $S \subseteq \mathbb{N}$ of *item sizes*, a number $k \in \mathbb{N}$ of (identical) *bins*, and a *maximum capacity* $M \in \mathbb{N}$ of each bin, whether the items will fit into the

bins without overflowing.

(*i*) Formulate this decision problem as language BINPACK.

(*ii*) Show that BINPACK $\in$ NP by providing a non-deterministic polynomial-time Turing machine decider.

(*iii*) Show that BINPACK $\in$ NP by providing a deterministic polynomial-time Turing machine verifier.

(*i*)

$$\text{BINPACK} = \left\{ \langle S,k,M \rangle \;\middle|\; \begin{array}{c} S \subseteq \mathbb{N},\, k \in \mathbb{N},\, M \in \mathbb{N}, \\ \exists S_1,\ldots,S_k \subseteq S.\, \bigcup_{i\in[k]} S_i = S, \\ \forall i,j \in [k].(i \neq j) \to S_i \cap S_j = \varnothing, \\ \forall i \in [k].\, \sum_{s\in S_i} s \leq M \end{array} \right\}$$

A partition of S into k parts is equivalent to a function $f : |S| \to [k]$ (assign $(i \in |S|)$th elemnt of S to the $(j \in [k])$th bin), so equivalently:

$$\text{BINPACK} = \left\{ \langle S,k,M \rangle \;\middle|\; \begin{array}{c} S \subseteq \mathbb{N},\, k \in \mathbb{N},\, M \in \mathbb{N}, \\ \exists f : S \to [k]\ \forall j \in [k].\, \sum_{s\in f^{-1}\left[\{j\}\right]} s \leq M \end{array} \right\}$$

(*ii*)

(1) Input $\langle S,k,M \rangle$.

(2) Non-deterministically split S into a partition $S_1,\ldots,S_k$.

(3) For $i = 1$ to k, compute $\sum_{s\in S_i} s$ and verify that each one is at most M.

(4) If any sum exceeds M, reject. Otherwise accept.

There are $O(|S|)$ additions performed, with each operation linear in the bit-length of the elements of S, and there are k-many checks at the end, each of which is linear in bit-length. All of these are (sub-)polynomial, so the algorithm is polynomial-time, and BINPACK $\in$ NP.

Computing the exact time complexity of this algorithm is slightly involved. Let $n = |S|$.

Firstly, guessing the partition is not constant time: interpreting a partition as a function $f : [n] \to [k]$, this requires writing down n integers, each $O(\log k)$ bits long, costing $O(n \log k)$ in total.

Now, let $m = \max_{s\in S} \lceil \log s \rceil$ be the largest bit-length of an element in S. Then, for each $s \in S$, $s < 2^m$, so

$$\sum_{s\in S} s < n \cdot 2^m \leq 2^{m+\log n}$$

so the bit-length of any summand is at most $O(m + \log n)$. There are n additions to compute, and addition is linear in bit-length, so this costs $O\big(n(m + \log n)\big)$ overall.

We then perform k-many comparisons, each costing $O\big(\max(m + \log n, \lceil \log M \rceil)\big)$, which is $O\big(k(m + \log n + \lceil \log M \rceil)\big)$ in total.

So this algorithm is $O\big(n \log k + n(m + \log n) + k(m + \log n + \lceil \log M \rceil)\big)$. In terms of the input length $L \in \Theta(nm + \log k + \log M)$, the algorithm takes $O(L^2 + L \log L) = O(L^2)$, which is polynomial. (Since $k \leq n$ for any non-trivial instance of bin-packing, this complexity simplifies to $O\big(n(m + \log n + \log k + \lceil \log M \rceil)\big) = O(L \log L)$ in practice.)

In any case, this

(*iii*) Steps 3 and 4 in the previous algorithm constitute a deterministic polynomial-time verification algorithm.

$Q5$

The following algorithms erroneously claim to decide PATH in non-deterministic polynomial-time. Find and explain the mistakes.

- Algorithm 1:
  (1) Input $\langle G,s,t \rangle$.
  (2) Non-deterministically select a number $k \in \mathbb{N}$.
  (3) Non-deterministically select a list of k vertices from G.
  (4) Verify that the first node is s, the last node is t, and each pair of adjacent nodes are connected by an edge in G.
  (5) If this is the case, accept. Otherwise reject.
- Algorithm 2:
  (1) Input $\langle G,s,t \rangle$ of length n.
  (2) Non-deterministically select a number $k \leq 2^n$.
  (3) Non-deterministically select a list of k vertices from G.
  (4) Verify that the first node is s, the last node is t, and each pair of adjacent nodes are connected by an edge in G.
  (5) If this is the case, accept. Otherwise reject.

(*i*) In step (2), the algorithm attempts to non-deterministically select a number $k \in \mathbb{N}$. However, Turing machines can only have finitely-many states, so the computation tree must be finitely-branching. This step, taken literally, is not finitely-branching, and thus does not describe a Turing machine at all.

This issue can be amended by guessing k bit-by-bit so that the branching is finite, but the machine could then generate arbitrarily large k on some branches – in particular, values of k that are not polynomially bounded in the size of the input.

(*ii*) This version attempts to patch this issue by bounding k by 2^n, but this bound is too permissive, and still allows super-exponential branches. In particular, steps 3 and 4 require looping over (at least) k-many vertices, so if $k = 2^n$, then this already takes $O(2^n)$ time to verify.

To fix the algorithm, we can simply bound k by $|V(G)|$. This is sufficient since there is no point visiting a vertex more than once when attempting to determine reachability. With this bound, representing k costs $O(\log n)$; writing the list of vertices costs $O(n \log n)$; and verifying the list costs $O(n)$.

Q6

Show that

$$\text{MODEXP} = \left\{\langle a,b,c,p\rangle \mid a,b,c,p \in \mathbb{Z}^+ \text{ and } a^b \equiv c \bmod p\right\}$$

is in P.

We cannot naïvely compute a^b and then reduce the result modulo p, because a^b is exponential in the size of b. Instead, we can compute the exponential in stages, and reduce modulo p at every step, thus bounding the size by p.

We can do this efficiently by using the binary representation of b. If $b = b_k b_{k-1} \cdots b_1 b_0$ is the binary representation of b, then

$$a^b = a^{\sum_{i=0}^{k-1} b_i 2^i} = \prod_{i=0}^{k-1} a^{b_i 2^i} = \prod_{i=0}^{k-1} (a^{2^i})^{b_i} = \prod_{b_i=1} a^{2^i}$$

That is, we can reconstruct a^b by computing $a^1, a^2, a^4, a^8, a^{16}, \ldots, a^{k-2}, a^{k-1}$, then multiplying the terms corresponding to non-zero bits in the binary expansion of b.

We can optimise this further by only holding one power of a at a time and accumulating the result of the partial computation at each step:

Algorithm 5 Decide modular exponentiation

```
procedure DECIDE(⟨a,b,c,p⟩)
    result ← 1
    base ← a mod p
    for i = 0 to ⌈log_2 b⌉ − 1 do
        if b_i = 1 then
            result ← (result · base) mod p
        end if
        base ← (base · base) mod p
    end for
    if result = c mod p then
        return accept
    else
        return reject
    end if
end procedure
```

There are $\lceil \log b \rceil$-many iterations in the for-loop, each of which requiring at most 2 modular multiplications. Since all of the numbers involved are bounded by p, multiplication is at most $O\big((\log p)^2\big)$ with classical algorithms,* and produces an integer of length at most $2 \log p$; computing the modular remainder of the result (i.e. integer division) then costs $O\big((2 \log p)^2\big)$. Finally, the equality check takes $O(\log p)$ time.

So the total running time is $O\big(\log b (\log p)^2\big)$. In terms of the input length $L \in \Theta(\log a + \log b + \log c + \log p)$, this algorithm is $O(L^3)$, which is polynomial.

*Modern algorithms can improve this to $O(\log p \log\log p)$ or better, but we just need any polynomial bound.

Q7

Show that the class P is closed under Kleene star.

Let $L \in \mathrm{P}$, and let M be a deterministic Turing machine deciding the membership of a word w of length n in L in $O(n^c)$.

Given a word $w \in L$, there are 2^{n-1} decompositions of w into substrings, so naïvely checking all possible decompositions of w is not viable. However, we can use dynamic programming and incrementally test membership of small substrings of w, memoising the results in a fast-access data structure to speed up later larger strings.

(1) Initialise a Boolean array $A[0..n]$.

We will fill in this array such that $A[i] = \texttt{true}$ if and only if $w[0..i-1] \in L^*$, so $w \in L^*$ if and only if $A[n] = \texttt{true}$.

(2) First, $A[0] = \texttt{true}$ because $w[0..-1] = \varepsilon$ is always in L^*.

(3) Then, for $i \in [1,n]$ and for $j \in [0,i-1]$, we split $w[0..i-1]$ at the jth position to obtain $w[0..j-1]$ and $w[j..i-1]$. The former is a string strictly shorter than $w[0..i-1]$ whose membership can be read from A, and the latter can be decided by M.

(4) If both checks return true, then set $A[i] = \texttt{true}$ and `break`.

(5) Accept if $A[n] = \texttt{true}$.

Algorithm 6 Decide Kleene star

```
procedure DECIDE(w)
    n ← |w|
    A[0..n] ← []
    A[0] ← true
    for i = 1 to n do
        A[i] ← false
        for j = 0 to i − 1 do
            if A[j] = true and M(w[j..i − 1]) then
                A[i] ← true
                break
            end if
        end for
    end for
    if A[n] = true then
        return accept
    else
        return reject
    end if
end procedure
```

The outer for-loop has $O(n)$ iterations, and the inner for-loop runs at most n times. Each check of $A[j]$ takes $O(1)$ time, and each query of M takes $O\big((i-j)^c\big) \subseteq O(n^c)$ time. So, the overall running time is $O(n^{c+2})$, which is polynomial.